

Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

April 29, 2026

- 1 Home
- 2 Open-FDD
 - 2.1 What it does
 - 2.2 Quick start
 - 2.3 Documentation
 - 2.4 License
- 3 System Overview
- 4 System overview
 - 4.1 Data flow
 - 4.2 Related topics
- 5 Column map & resolvers
- 6 Column map & resolvers
 - 6.1 1. Plain dict (most IoT / warehouse flows)
 - 6.2 2. Brick TTL (your integration)
 - 6.3 3. Manifest file (ManifestColumnMapResolver)
 - 6.4 4. Composite priority (FirstWinsCompositeResolver)
 - 6.5 Examples in the repo
 - 6.6 See also
- 7 Getting Started
- 8 Getting Started
 - 8.1 Install from PyPI
 - 8.2 Develop from a git checkout
 - 8.3 Examples
 - 8.4 Where to read next
- 9 Modular Architecture
- 10 Modular architecture
 - 10.1 Layers
 - 10.2 Extension points
 - 10.3 See also
- 11 Field data and BACnet-shaped metadata
- 12 Field data and BACnet-shaped metadata
- 13 Data modeling & platform docs
- 14 Data modeling and full-stack docs
 - 14.1 In *this* repository (rules engine only)
 - 14.2 AI-assisted modeling workflows
 - 14.3 Desktop bridge note (open-fdd repo)
- 15 Overview

- 16 Fault rules
 - 16.1 Where rules live
 - 16.2 Hot reload
 - 16.3 Rule types
 - 16.4 YAML structure
 - 16.5 Running rules
 - 16.6 Engineering metadata and analytics
 - 16.7 Cookbook
- 17 Expression Rule Cookbook
- 18 Expression Rule Cookbook
 - 18.1 Ontology labels (Brick, Haystack, DBO, 223P)
 - 18.2 Occupied hours and weather gating (expressions)
 - 18.3 How to define expressions
 - 18.4 Signal scaling (0-1 fraction vs 0-100 percent)
 - 18.5 AHU rules (reference-style)
 - 18.5.1 Rule A — Duct static below setpoint at full fan speed
 - 18.5.2 Rule B — Blended air temp below expected band
 - 18.5.3 Rule C — Blended air temp above expected band
 - 18.5.4 Rule D — Discharge air cold when heating commanded
 - 18.5.5 Rule E — SAT too low with full heating
 - 18.5.6 Rule F — SAT/MAT mismatch in economizer mode
 - 18.5.7 Rule G — Ambient too warm for free cooling
 - 18.5.8 Rule H — Ambient vs blended mismatch (econ + mech cooling)
 - 18.5.9 Rule I — Ambient vs blended mismatch (econ-only)
 - 18.5.10 Rule J — Discharge above blended in cooling
 - 18.5.11 Rule K — Discharge above setpoint in full cooling
 - 18.5.12 Rule L — Cooling coil delta-T when inactive
 - 18.5.13 Rule M — Heating coil delta-T when inactive
 - 18.6 Central plant
 - 18.6.1 Differential pressure at max pump speed
 - 18.6.2 Plant flow high at max pump
 - 18.6.3 Plant supply temp outside deadband
 - 18.6.4 Chiller runtime over daily limit
 - 18.7 Heat pumps
 - 18.7.1 Discharge cold when heating
 - 18.8 VAV zones
 - 18.9 Standard AHU/VAV starter pack (practical baseline)
 - 18.9.1 Starter: economizer 100% OA temperature tracking
 - 18.9.2 Starter: mechanical cooling when free cooling should be used
 - 18.9.3 Starter: OA damper too open in hot (OAT > 70 F / 21.1 C) or cold (OAT < 40 F / 4.4 C) ambient
 - 18.9.4 Metric equivalents quick note
 - 18.9.5 Excessive heating during warm weather
 - 18.9.6 Damper or valve at full open
 - 18.9.7 Zone and IAQ bounds
 - 18.10 Opportunistic rules (economizer & ventilation)
 - 18.10.1 Economizing when outdoor conditions are unfavorable
 - 18.10.2 Mechanical cooling when econ could suffice

- 18.10.3 Low ventilation (estimated OA fraction)
- 18.10.4 Preheat over-conditioning
- 18.10.5 Blended air damper deviation
- 18.11 Weather station
 - 18.11.1 Unrealistic temperature spike
 - 18.11.2 RH bounds
 - 18.11.3 Wind gust vs sustained
- 18.12 Sensor validation (bounds & flatline)
- 18.13 Follow-up (RFCs): multi-ontology selectors & cookbook matrix
- 19 Test bench rule catalog
- 20 Rule YAML catalog (in this repository)
 - 20.1 open_fdd/tests/fixtures/rules/ (pytest)
 - 20.2 examples/AHU/rules/ (notebooks / demos)
 - 20.3 Related docs
- 21 YAML rules → Pandas (under the hood)
- 22 YAML rules → Pandas DataFrames (under the hood)
 - 22.1 1. YAML never becomes “one giant DataFrame of rules”
 - 22.2 2. From long telemetry rows to a wide DataFrame
 - 22.3 3. What RuleRunner.run does to that DataFrame
 - 22.4 4. How a YAML rule dict becomes pandas expressions
 - 22.5 5. Reloading rules vs DataFrame lifetime
 - 22.6 6. Mental model checklist
- 23 Fault rules for HVAC
- 24 Fault rules for HVAC
- 25 Security
- 26 Security
 - 26.1 Supply chain and dependencies
 - 26.2 Rule YAML and expressions
 - 26.3 Reporting issues
- 27 Cloud export example
- 28 Cloud export example
 - 28.1 What the example does
 - 28.2 Run the example
 - 28.3 Use it as your starting point
- 29 RDF, Brick, and optional BACnet-shaped metadata
- 30 RDF, Brick, and optional BACnet-shaped metadata
- 31 Context and recordkeeping
- 32 Context and recordkeeping
 - 32.1 Rule
 - 32.2 Good context to commit
 - 32.3 Context anti-patterns
- 33 Operational states
- 34 Operational states
 - 34.1 States
 - 34.2 Why this matters
- 35 Concepts
- 36 Concepts
- 37 How-to Guides
- 38 How-to Guides
 - 38.1 Engine
- 39 PyPI releases (open-fdd)

- 40 PyPI releases (open-fdd)
 - 40.1 openfdd-engine (optional second PyPI name)
 - 40.2 Release baseline (PyPI)
 - 40.3 1) Before a release of open-fdd
 - 40.3.1 Transition checklist (legacy 0.1.x → 2.x on PyPI)
 - 40.3.2 PyPI upload auth (open-fdd only)
 - 40.3.3 Fallback: API token
 - 40.3.4 If CI shows invalid-publisher or 403
 - 40.4 2) Publish open-fdd (commands)
 - 40.4.1 Local dry run
 - 40.4.2 GitHub Actions
 - 40.5 3) Publish openfdd-engine (second PyPI project)
 - 40.6 Scope policy
- 41 Mode-aware runbooks
- 42 Mode-aware runbooks
 - 42.1 Recommended checks
 - 42.2 Test bench vs production data
- 43 Testing plan
- 44 Testing plan
 - 44.1 Continuous integration
 - 44.2 Local hygiene
 - 44.3 Optional benches
- 45 Operations
- 46 Operations
- 47 Configuration
- 48 Configuration
 - 48.1 Rule YAML
 - 48.2 Column maps
 - 48.3 Environment variables
 - 48.4 Related topics
- 49 Contributing
- 50 Contributing to Open-FDD
 - 50.1 Table of Contents
 - 50.2 Code of Conduct
 - 50.3 I Have a Question
 - 50.4 I Want To Contribute
 - 50.4.1 Legal notice
 - 50.5 Reporting Bugs
 - 50.5.1 Before submitting a bug report
 - 50.5.2 How to submit a good bug report
 - 50.6 Suggesting Enhancements
 - 50.6.1 Before submitting
 - 50.6.2 How to submit a good enhancement suggestion
 - 50.7 Contributing Rules (Expression Rule Cookbook)
 - 50.8 Your First Code Contribution
 - 50.9 Improving the Documentation
 - 50.10 Styleguides
 - 50.11 Git workflow (branch → PR → sync)
 - 50.12 Commit Messages
 - 50.13 Join the Project Team
 - 50.14 Attribution
- 51 AI PR review playbook

- 52 AI PR review playbook
 - 52.1 Priority order
 - 52.2 Review output contract
- 53 Appendix
- 54 Appendix
- 55 Cloning and porting
- 56 Cloning and porting
 - 56.1 What transfers cleanly
 - 56.2 What changes per deployment
 - 56.3 Recommended first pass on a new machine
 - 56.4 Portability goal
- 57 Verification
- 58 Verification
 - 58.1 1. Unit tests (CI)
 - 58.2 2. Rule YAML sanity
 - 58.3 3. Small DataFrame smoke test
 - 58.4 Platform / database checks
- 59 Engine-only deployment and external IoT pipelines
- 60 Engine-only deployment and external IoT pipelines
 - 60.1 Library path — same YAML, any DataFrame
 - 60.1.1 Bring your own `column_map` or resolver
 - 60.1.2 Manifest file + composite priority
 - 60.2 Standalone playground (optional)
 - 60.3 Summary
- 61 The optional `openfdd-engine` package
- 62 The optional `openfdd-engine` package (`openfdd_engine`)
 - 62.1 Re-exports only
 - 62.2 Mental model
 - 62.3 When to use `pip install open-fdd` vs `openfdd-engine`
 - 62.4 Summary
 - 62.5 See also
- 63 Desktop App
- 64 Open-FDD Desktop App
 - 64.1 Goal
 - 64.2 Install
 - 64.3 Launch
 - 64.3.1 Container-friendly web launch (bridge + MCP + React UI)
 - 64.3.2 Desktop bridge Swagger/OpenAPI
 - 64.4 MCP RAG service (OpenClaw/agents)
 - 64.5 Data ingest quickstart (desktop bridge API)
 - 64.5.1 1) Create or list sites
 - 64.5.2 2) CSV ingest (file path or upload)
 - 64.5.3 3) Open-Meteo weather ingest
 - 64.5.4 4) Onboard ingest
 - 64.5.5 5) BACnet ingest (one-shot) and polling
 - 64.5.6 6) Join sources for analysis/plots
 - 64.6 CSV timestamp parsing expectations
 - 64.7 Data model + BRICK
 - 64.8 Feather-first ingestion
 - 64.9 Rule loop behavior
 - 64.10 Typical desktop workflow (current)

1 Home

2 Open-FDD

The **open-fdd** package (`open_fdd/` in this repository) is a **rules engine** for building science and HVAC workflows: you define fault checks in **YAML**, map columns on a **pandas** DataFrame, and run them with **RuleRunner**. It is published to **PyPI** as a small, dependency-conscious wheel (pandas, NumPy, PyYAML, pydantic).

Full platform (data model, APIs, Docker, Brick/223P tooling beyond `column_map`): **open-fdd-afdd-stack — docs/** — it uses this library under the hood.

2.1 What it does

- **Loads** rule definitions from YAML (bounds, flatline, hunting, expressions, schedules, weather gates, ...).
- **Maps Brick, Haystack, DBO, 223P**, or vendor names to DataFrame columns via `column_map` (dict, manifest, or composite resolvers) — same YAML can target different ontologies by swapping the map.
- **Runs** checks over time-indexed or ordered data and returns structured **fault results** (see `open_fdd.schema`).

Desktop mode is also available and under active construction in this repository. The current desktop path uses local Feather-backed ingestion plus optional batched rule execution for large datasets.

Bring your own data: CSV exports, historian extracts, lab benches, or notebooks. The engine does **not** connect to databases or field buses by itself.

2.2 Quick start

```
pip install open-fdd
```

From a git checkout:

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
pytest
```

See **Getting started** and **examples/README.md** in the repository for runnable entrypoints.

2.3 Documentation

Section	Description
Getting started	Install, tests, examples
Rules overview	Rule types and YAML structure
Expression rule cookbook	Expressions, ontology labels, schedule & weather gates
Column map resolvers	Brick / Haystack / DBO / 223P → columns
Engine API	RuleRunner, loaders, resolvers
Desktop app (under construction)	Local desktop workflow, Feather storage, batched rule runs
Data modeling & platform (pointer)	Full stack docs live in open-fdd-afdd-stack
How-to guides	PyPI releases, verification, operations
Appendix	Technical reference, developer guide

2.4 License

MIT — see the repository **LICENSE** file.

3 System Overview

4 System overview

Open-FDD (package name `open-fdd`, import `open_fdd`) is a **library** for running **fault detection and diagnostics (FDD)** rules on **pandas** DataFrames. Rules are authored in **YAML**; the core type is **RuleRunner** in `open_fdd.engine`.

4.1 Data flow

1. **Your pipeline** loads or builds a time-indexed (or otherwise keyed) DataFrame of sensor or calculated points.
2. **Column mapping** connects logical point names used in rules to actual column names (dict, manifest YAML, or custom resolver).
3. **RuleRunner** evaluates configured checks and returns **fault results** compatible with `open_fdd.schema`.

There is no required database, HTTP service, or message bus in this repository—the engine runs wherever you import it.

4.2 Related topics

- [Modular architecture](#) — how rule types and resolvers fit together
- [Rules overview](#)
- [Engine API](#)

5 Column map & resolvers

6 Column map & resolvers

YAML rules name their **inputs** with **logical keys**—often **Brick** class names (e.g. `Supply_Air_Temperature_Sensor`), but the same **column_map** mechanism supports **Haystack** refs, **DBO** types,

223P / **s223** strings, or any custom key you put on the input dict. Your **pandas** frame uses **your** column names (sat, AHU1_SupplyTemp, ...). See [Expression rule cookbook — ontology labels](#).

column_map is the bridge: dict[str, str] where each **key** is a **logical label** (Brick class, or a string from the rule's optional **haystack** / **dbo** / **s223** / **223p** fields on that input), and each **value** is the **actual DataFrame column name**. RuleRunner tries those fields in order until one matches a map key; see [Expression Rule Cookbook](#).

Pass it to **RuleRunner.run(..., column_map=...)**. You can build that dict by hand, load it from a manifest, or **merge** several sources with a composite resolver. Deriving the map from a Brick **.ttl** file with SPARQL is done in **your** services (typically with **rdflib** installed there), not inside the **open-fdd** wheel.

6.1 1. Plain dict (most IoT / warehouse flows)

```
runner.run(
    df,
    column_map={
        "Supply_Air_Temperature_Sensor": "sat",
        "Outside_Air_Temperature_Sensor": "oat",
    },
)
```

No TTL file required if you already know the mapping. Keys must match the **input names** used in your rule YAML (see [Fault rules overview](#)).

6.2 2. Brick TTL (your integration)

If you maintain **data_model.ttl** (or another graph), run SPARQL or graph walks in **your** Python environment (**rdflib**, etc.) and reduce the result to the same dict[str, str] you would pass to **RuleRunner.run**.

For **engine-only** notebooks and pipelines, a **manifest** (below) or **plain dict** is usually simpler.

6.3 3. Manifest file (ManifestColumnMapResolver)

JSON or **YAML**: either a flat `logical_name → column_name` object, or a top-level `column_map`: nested mapping.

```
from pathlib import Path
from open_fdd.engine.column_map_resolver import
ManifestColumnMapResolver

r = ManifestColumnMapResolver(Path("my_mapping.yaml"))
runner.run(df, column_map=r.build_column_map(ttl_path=Path(".")))
```

For a raw dict without the resolver class, use `load_column_map_manifest(path)` from the same module.

6.4 4. Composite priority (FirstWinsCompositeResolver)

Run multiple resolvers **in order**; for each logical key, the **first resolver that defines it wins** (for example: a base manifest, then an override file that only adds missing keys). On the stack you can compose the stack's **BrickTtlColumnMapResolver** with a **ManifestColumnMapResolver** in your own startup code.

```
from pathlib import Path
from open_fdd.engine.column_map_resolver import (
    FirstWinsCompositeResolver,
    ManifestColumnMapResolver,
)

composite = FirstWinsCompositeResolver(
    ManifestColumnMapResolver("base.yaml"),
    ManifestColumnMapResolver("extras.yaml"),
)
runner.run(df,
column_map=composite.build_column_map(ttl_path=Path(".")))
```

Security: There is **no** env-driven “import a resolver class by string” — compose resolvers in code or a small startup script.

6.5 Examples in the repo

Minimal workshop — `examples/column_map_resolver_workshop/`
([folder on GitHub](#)):

Resource	Link
Run	simple_ontology_demo.py
Rule YAML (cookbook-style inputs)	simple_ontology_rule.yaml
Walkthrough	README.md
Top-level index	examples/README.md

AHU / RTU notebooks and sample rules — **examples/AHU/** ([folder on GitHub](#)):

Resource	Link
Sample YAML rules	rules/sensor_bounds.yaml , rules/sensor_flatline.yaml , rules/sat_operating_band.yaml
Notebooks	RTU11_standardized_refactored.ipynb , RTU7_standardized_refactored.ipynb
Sample CSV	RTU11.csv , AHU7.csv
Helpers	openfdd_notebook_helpers_v2.py

More narrative (IoT pipelines, **column_map policy**): [Engine-only & IoT pipelines](#). Curated doc links: [Examples \(repository\)](#).

6.6 See also

- [Getting started](#) — **RuleRunner** + **column_map** in one example
- [Engine-only & IoT pipelines](#) — integration patterns and resolver priority
- [Fault rules overview](#) — YAML **inputs** and Brick-oriented keys
- [Expression rule cookbook](#) — expression rules (same keying model)
- [Examples \(repository\)](#) — notebooks and CSVs under **examples/AHU/**

7 Getting Started

8 Getting Started

Use this page to install **open-fdd**, run the test suite from a checkout, and find examples.

8.1 Install from PyPI

```
pip install open-fdd
```

Python **3.9+** is supported (see `pyproject.toml` classifiers).

8.2 Develop from a git checkout

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
python3 -m venv .venv
source .venv/bin/activate #
Windows: .venv\Scripts\activate
pip install -U pip
pip install -e ".[dev]"
python -c "import open_fdd; print('open_fdd import OK')"
pytest
```

8.3 Examples

See `examples/README.md` in the repository for CSV-driven demos, ontology examples, and notebooks. Many examples assume **pandas** only; optional plotting or Word output need extra packages.

8.4 Where to read next

- [Rules overview](#)
- [Column map resolvers](#)
- [Engine API](#)
- [How-to: engine-only IoT](#)
- [Contributing](#)

9 Modular Architecture

10 Modular architecture

This document describes how the `open_fdd` package is structured at a high level. The goal is a **small core** (rules + checks + column mapping) that you can embed in notebooks, batch jobs, or larger applications.

10.1 Layers

Layer	Role
Rule YAML	Human-authored definitions (bounds, flatline, expression, ...) loaded by <code>load_rule</code> / <code>RuleRunner</code> .
Column map	Maps logical names to DataFrame columns (ColumnMapResolver, manifest YAML, composite resolvers).
Checks	Pure functions over Series/DataFrames (bounds, rate of change, expressions with safe eval, ...).
Runner	Orchestrates checks, schedules, and optional weather or derived columns.
Schema	pydantic models for outputs (fault codes, intervals, metadata).

10.2 Extension points

- **Custom resolvers** — implement the column-map resolver protocol for site-specific naming.
 - **Expression rules** — use the documented expression language and built-in helpers (see [Expression rule cookbook](#)).
 - **Reports** — optional modules under `open_fdd.reports` for visualization or document export (may need extra dependencies).
-

10.3 See also

- [Technical reference](#)
- [Column map resolvers](#)

11 Field data and BACnet-shaped metadata

12 Field data and BACnet-shaped metadata

The **open-fdd** engine consumes **pandas** DataFrames. It does **not** speak BACnet, Modbus, or other field protocols.

If your integration maps BACnet object names (or similar) into column names or Brick-style identifiers, use **column_map** manifests or a custom resolver—see [Column map resolvers](#) and [RDE, Brick, and optional BACnet-shaped metadata](#).

For BACnet **discovery or control**, use your building's **gateway, BMS, or integration stack**, then pass normalized data into the engine.

13 Data modeling & platform docs

14 Data modeling and full-stack docs

Brick, 223P, SPARQL, CRUD APIs, Docker Compose, and lab automation for Open-FDD as a **deployed platform** now live in a separate repository:

[open-fdd-afdd-stack — docs/](#)

That site documents how the stack uses this **open-fdd** PyPI package **under the hood** (RuleRunner, YAML rules, pandas).

14.1 In *this* repository (rules engine only)

- **Column map resolvers** — map **Brick**, **Haystack**, **DBO**, **223P**, or vendor labels to DataFrame columns (dict, manifest, composite resolvers).
- **Expression rule cookbook** — fault logic on pandas, including schedule and weather gates via `params.schedule` / `params.weather_band`.
- `examples/column_map_resolver_workshop/` — runnable **ontology-agnostic** demo (`simple_ontology_demo.py`).

Semantic modeling and TTL/SQL integration are **not** part of the **open-fdd** wheel; keep RDF/graph work in your services or in **open-fdd-afdd-stack** as documented there.

14.2 AI-assisted modeling workflows

For AI-assisted data modeling (OpenClaw, ChatGPT, or human-in-the-loop review), use a simple loop:

1. Export model JSON from your backend (`/model/export` or stack export endpoint).
2. Review and revise with an LLM (OpenClaw agent or ChatGPT web UI).
3. Validate the edited JSON before import.
4. Import JSON back to backend (`/model/import`) and re-run SPARQL/rules checks.

The same flow works for:

- **OpenClaw agents** running local automation loops.
- **ChatGPT online interface** where a human copies export JSON in and validated JSON out.
- **Hybrid** workflows where AI drafts and human confirms before import.

For robust prompts, import schema guidance, and operator-safe pre-flight checks, see:

- [**AFDD stack modeling docs**](#)
 - [**LLM workflow page**](#)
-

14.3 Desktop bridge note (open-fdd repo)

The desktop bridge now supports agent-friendly backend operations such as:

- model export/import/validate,
- SPARQL query endpoints (/data-model/sparql, /data-model/sparql/upload),
- timeseries bounds/query over Feather data,
- weather/onboard ingest and ML training routes.

This enables OpenClaw-style local assistants to do data modeling, retrieve and join data in pandas/Feather workflows, run faults, and iterate with a human operator.

15 Overview

16 Fault rules

Fault rules are **YAML-defined** checks run against **pandas** DataFrames. Each rule produces boolean fault flag columns (and related outputs) via **RuleRunner**.

Rules often use **Brick class names** as logical input keys; you supply a **column_map** from those keys to your actual DataFrame columns (dict, manifest, or custom resolver). See [Column map resolvers](#) and the [Expression rule cookbook](#).

16.1 Where rules live

Keep rule YAML in a **single directory** (for example `my_rules/`) or pass parsed dicts to **RuleRunner(rules=[...])**. Example snippets live under **examples/** and **open_fdd/tests/fixtures/rules/** in this repository.

See the [Expression rule cookbook](#) to add or adapt rules. For how YAML becomes pandas operations, see [YAML rules → Pandas \(under the hood\)](#).

16.2 Hot reload

RuleRunner reads the rule list you give it at construction time. To pick up disk edits, call `load_rules_from_dir` again (or construct a new **RuleRunner**) before the next `run()`.

16.3 Rule types

Type	Purpose
bounds	Value outside [low, high]
flatline	Rolling spread < tolerance (stuck sensor)
hunting	Excessive state changes (PID hunting)
expression	Custom pandas/numpy expression; supports schedule/weather gating via <code>params.schedule</code> and <code>params.weather_band</code>
oa_fraction	OA fraction vs design airflow error
erv_efficiency	ERV effectiveness out of range

These six values are the built-in type values in the engine.

Occupied-hours and weather gating are not separate rule types; they are optional masks injected for type: expression rules (`schedule_occupied`, `weather_allows_fdd`). See the [Expression rule cookbook](#).

16.4 YAML structure

Example with Brick-style logical keys (map them with `column_map`):

```
name: sensor_bounds
type: bounds
flag: bad_sensor
inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor
params:
  low: 40
  high: 90
```

```
        rolling_window: 6    # optional: require N consecutive True
                               samples before flagging
```

Per-rule rolling window: Set `params.rolling_window` (e.g. 6) in a rule to require that many consecutive True samples before the fault is flagged. Omit for “flag on any True.” See fixtures such as `sensor_flatline.yaml` under `open_fdd/tests/fixtures/rules/`.

16.5 Running rules

```
from pathlib import Path
from open_fdd.engine.runner import RuleRunner

runner = RuleRunner(rules_path=Path("path/to/rules"))
out = runner.run(df, timestamp_col="timestamp", column_map={...})
```

16.6 Engineering metadata and analytics

Downstream analytics often combine **fault flags** from `RuleRunner` with **equipment metadata** and **time series** you already store. For Brick / 223P graph and API workflows, see [open-fdd-afdd-stack docs](#) and [Data modeling & platform docs](#).

16.7 Cookbook

See [Expression rule cookbook](#) for AHU, chiller, weather, and advanced recipes.

17 Expression Rule Cookbook

18 Expression Rule Cookbook

A reference for building fault detection rules in open-fdd. Rules use **YAML** with **expression** type: when the expression evaluates to **True**, a fault is flagged. open-fdd injects **NumPy** as `np` into expression evaluation, so you can use `np.maximum`, `np.abs`, `np.sqrt`,

etc. for vectorized math. **Tuning:** Change params in YAML and re-run **RuleRunner** on your DataFrame. See [Fault rules overview](#) and [Configuration](#).

column_map: The engine is **ontology-agnostic**: YAML inputs expose logical names you use in the **expression**. **column_map** (dict or manifest) maps **Brick, Haystack, DBO, 223P** (s223 / 223p fields), or arbitrary keys → **actual DataFrame columns**. Optional per-input fields **brick, haystack, dbo, s223, 223p** are tried **in that order** against **column_map** (**first match wins**). See [Column map resolvers](#) and [examples/column_map_resolver_workshop/simple_ontology_demo.py](#). Full **graph, TTL, SPARQL, and CRUD** workflows live in **[open-fdd-afdd-stack docs](#)**.

18.1 Ontology labels (Brick, Haystack, DBO, 223P)

Most cookbook recipes use **Brick class names** because that is a common **interoperable** vocabulary—but the **same rule file** can run against a Haystack- or 223P-keyed **column_map** if your inputs declare the matching field:

```
inputs:
  sat:
    brick: Supply_Air_Temperature_Sensor
    haystack: "your-haystack-ref"
    dbo: "your-dbo-type"
    s223: "your-s223-term"
```

Pass a **column_map** whose keys are exactly those strings (or use **ManifestColumnMapResolver** / **FirstWinsCompositeResolver**). Workshop: [examples/column_map_resolver_workshop/README.md](#).

Brick + timeseries metadata: If you follow [Brick external references](#) in a graph pipeline, you still reduce that to a **dict[str, str]** before calling **RuleRunner** here.

18.2 Occupied hours and weather gating (expressions)

For **type: expression**, optional **params.schedule** defines weekday + hour-of-day occupancy. The engine injects boolean **schedule_occupied** (True **inside** the window). Combine with your equipment logic, e.g. **fan running when unoccupied**:

```

params:
  schedule:
    weekdays: [0, 1, 2, 3, 4]  # Mon–Fri; Mon=0 ... Sun=6
    start_hour: 8
    end_hour: 17               # 8:00–16:59 local hour
  weather_band:
    oat_input: Outside_Air_Temperature_Sensor
    units: imperial
    low: 32
    high: 85

expression: |
  fan_on & ~schedule_occupied & weather_allows_fdd

```

weather_band injects **weather_allows_fdd** (outside air in range). Omit nested blocks or set **enabled: false** to leave a mask all-True.

18.3 How to define expressions

1. **Inputs** — Each key becomes an alias in the expression. Prefer structured dicts with **brick** / **haystack** / **dbo** / **s223** / **223p** plus optional **column** (fallback DataFrame column when no **column_map** match).
2. **Params** — Thresholds and constants go in params. Reference by name (e.g. **err_thresh**, **vfd_max**).
3. **Expression** — Must evaluate to a boolean Series (True = fault). Use **&** (AND), **|** (OR), **~** (NOT). Use **.diff()**, **.rolling()**, **.notna()** for time-series logic.

Minimal example (column_map from Brick labels):

```

name: high_temp_check
type: expression
flag: high_temp_flag

inputs:
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor

params:
  max_temp: 90.0

expression: |
  Supply_Air_Temperature_Sensor > max_temp

runner.run(
  df,
  column_map={"Supply_Air_Temperature_Sensor": "SAT"},

```

```
        timestamp_col="timestamp",  
    )
```

18.4 Signal scaling (0-1 fraction vs 0-100 percent)

Many cookbook thresholds (e.g. `drv_hi_frac: 0.93`, comparisons to `0.01`) assume **command signals are in 0-1** (fraction of full scale). In the field, BACnet and some historians often expose **0-100** (percent) for the same semantics.

- **type: expression rules** use the DataFrame columns as-is. If your VFD command is 0-100 but the YAML uses `0.93`, either fix the data at the source, document units in the data model, or **scale in the expression**.
- **Built-in** rule types **hunting** and **oa_fraction** automatically treat signals as percent when **any** sample in the series is `> 1` (they divide by 100). **expression** rules do **not** apply that heuristic unless you do it explicitly.

Recommended patterns in expressions

1. **`normalize_cmd(series)`** (open-fdd 2.3+) — Injected next to `np`. Same heuristic as `hunting/oa_fraction`: if any finite value is `> 1`, the whole series is divided by 100; otherwise left as-is. Non-numeric cells become `NaN`.

```
(normalize_cmd(Supply_Fan_Speed_Command) >= drv_hi_frac -  
drv_near_hi)
```

2. **Explicit `np.where`** (works on all open-fdd versions):

```
(np.where(Supply_Fan_Speed_Command > 1,  
Supply_Fan_Speed_Command / 100.0, Supply_Fan_Speed_Command) >=  
0.93)
```

Mixed 0-1 and 0-100 in one series is ambiguous—prefer consistent scaling per point in your **source data** or **trend export**.

Engine validation (open-fdd 2.3+)

- Merged rule **params** are coerced/validated with **Pydantic** for `schedule` and `weather_band` (string numerics for scalars are best-effort coerced).
- **`RuleRunner.run(..., input_validation='warn'|'strict')`** checks that mapped columns exist and are **largely numeric** after `pd.to_numeric`; **strict** raises before evaluating expressions.
- In **strict** deployments, use **`RuleRunner.run(..., input_validation='strict', skip_missing_columns=False)`** with

open-fdd ≥ 2.3 so bad column maps fail fast before expressions run.

When `skip_missing_columns=True` (production default), rules that hit missing names/columns are **skipped** and a **warning** is logged (rule name, flag, exception).

18.5 AHU rules (reference-style)

The following rules follow common industry practice for air-handling fault detection. Rules A through M are adapted from ASHRAE Guideline 36 (GL36) AFDD guidance. Thresholds and logic are tunable; adjust params for your site.

18.5.1 Rule A — Duct static below setpoint at full fan speed

Static pressure under setpoint while supply fan runs near maximum. May indicate duct leakage, undersized fan, or terminal damper issues. *Adapted from GL36 AFDD guidance.*

```
name: duct_static_low_at_full_speed
description: Static pressure below setpoint when fan at full speed
(GL36-inspired)
type: expression
flag: rule_a_flag
equipment_type: [VAV_AHU]

inputs:
  Supply_Air_Static_Pressure_Sensor:
    brick: Supply_Air_Static_Pressure_Sensor
  Supply_Air_Static_Pressure_Setpoint:
    brick: Supply_Air_Static_Pressure_Setpoint
  Supply_Fan_Speed_Command:
    brick: Supply_Fan_Speed_Command

params:
  sp_margin: 0.12
  drv_hi_frac: 0.93
  drv_near_hi: 0.06

expression: |
  (Supply_Air_Static_Pressure_Sensor <
  Supply_Air_Static_Pressure_Setpoint - sp_margin) &
  (Supply_Fan_Speed_Command >= drv_hi_frac - drv_near_hi)
```

18.5.2 Rule B — Blended air temp below expected band

Blended air temp should lie between outdoor and return. If below both (minus tolerance), suspect sensor or mixing fault. *Adapted from GL36 AFDD guidance.*

```
name: blend_temp_below_band
description: Blended air temp below expected range (OAT/RAT)
(GL36-inspired)
type: expression
flag: rule_b_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Mixed_Air_Temperature_Sensor:
    brick: Mixed_Air_Temperature_Sensor
  Return_Air_Temperature_Sensor:
    brick: Return_Air_Temperature_Sensor
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Supply_Fan_Speed_Command:
    brick: Supply_Fan_Speed_Command

params:
  blend_tol: 1.15
  rat_tol: 1.15
  oat_tol: 1.15

expression: |
  (Mixed_Air_Temperature_Sensor - blend_tol <
np.minimum(Return_Air_Temperature_Sensor - rat_tol,
Outside_Air_Temperature_Sensor - oat_tol)) &
(Supply_Fan_Speed_Command > 0.01)
```

18.5.3 Rule C — Blended air temp above expected band

Blended air temp above the higher of OAT and RAT (plus tolerance) indicates mixing or sensor fault. *Adapted from GL36 AFDD guidance.*

```
name: blend_temp_above_band
description: Blended air temp above expected range (OAT/RAT)
(GL36-inspired)
type: expression
flag: rule_c_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Mixed_Air_Temperature_Sensor:
```

```

    brick: Mixed_Air_Temperature_Sensor
Return_Air_Temperature_Sensor:
    brick: Return_Air_Temperature_Sensor
Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
Supply_Fan_Speed_Command:
    brick: Supply_Fan_Speed_Command

params:
    blend_tol: 1.15
    rat_tol: 1.15
    oat_tol: 1.15

expression: |
    (Mixed_Air_Temperature_Sensor - blend_tol >
np.maximum(Return_Air_Temperature_Sensor + rat_tol,
Outside_Air_Temperature_Sensor + oat_tol)) &
(Supply_Fan_Speed_Command > 0.01)

```

Hunting/oscillation — see [rule types \(hunting\)](#).

18.5.4 Rule D — Discharge air cold when heating commanded

Discharge air temp below blended air when heating valve is open. Indicates heating coil or valve failure. *Adapted from GL36 AFDD guidance.*

```

name: discharge_cold_when_heating
description: Discharge air below blended air when heating active
(GL36-inspired)
type: expression
flag: rule_d_flag
equipment_type: [AHU, VAV_AHU]

inputs:
    Mixed_Air_Temperature_Sensor:
        brick: Mixed_Air_Temperature_Sensor
    Supply_Air_Temperature_Sensor:
        brick: Supply_Air_Temperature_Sensor
    Valve_Command:
        brick: Valve_Command
    Supply_Fan_Speed_Command:
        brick: Supply_Fan_Speed_Command

params:
    blend_tol: 1.15
    sat_tol: 1.15
    fan_delta_t: 0.55

expression: |

```

```
(Supply_Air_Temperature_Sensor + sat_tol <=
Mixed_Air_Temperature_Sensor - blend_tol + fan_delta_t) &
(Valve_Command > 0.01) & (Supply_Fan_Speed_Command > 0.01)
```

OA fraction — see [rule types \(oa_fraction\)](#).

18.5.5 Rule E — SAT too low with full heating

Heating valve fully open but SAT remains below setpoint. Indicates undersized coil or valve failure. *Adapted from GL36 AFDD guidance.*

```
name: sat_too_low_full_heating
description: SAT below setpoint with heating valve fully open
(GL36-inspired)
type: expression
flag: rule_e_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor
  Supply_Air_Temperature_Setpoint:
    brick: Supply_Air_Temperature_Setpoint
  Valve_Command:
    brick: Valve_Command
  Supply_Fan_Speed_Command:
    brick: Supply_Fan_Speed_Command

params:
  supply_err_thres: 1.0

expression: |
  (Supply_Air_Temperature_Sensor < Supply_Air_Temperature_Setpoint
- supply_err_thres) & (Valve_Command > 0.9) &
(Supply_Fan_Speed_Command > 0)
```

18.5.6 Rule F — SAT/MAT mismatch in economizer mode

In economizer mode (min mechanical cooling), SAT should approximate MAT. Large deviation suggests coil bypass or sensor error. *Adapted from GL36 AFDD guidance.*

```
name: discharge_blend_mismatch_econ
description: Discharge and blended air diverge in economizer mode
(GL36-inspired)
type: expression
flag: rule_f_flag
equipment_type: [AHU, VAV_AHU]
```

```

inputs:
  Mixed_Air_Temperature_Sensor:
    brick: Mixed_Air_Temperature_Sensor
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor
  Damper_Position_Command:
    brick: Damper_Position_Command
  Valve_Command:
    brick: Valve_Command

params:
  fan_delta_t: 0.55
  blend_tol: 1.15
  sat_tol: 1.15
  econ_min_open: 0.12

expression: |
  (np.abs(Supply_Air_Temperature_Sensor - fan_delta_t -
Mixed_Air_Temperature_Sensor) > np.sqrt(sat_tol**2 +
blend_tol**2)) & (Damper_Position_Command > econ_min_open) &
(Valve_Command < 0.1)

```

18.5.7 Rule G — Ambient too warm for free cooling

Outside air temperature exceeds SAT setpoint while economizer is active and mechanical cooling is off. Economizer should not be providing “free” cooling under these conditions. *Adapted from GL36 AFDD guidance.*

```

name: ambient_warm_free_cool
description: Outdoor air above setpoint in free cooling mode
(GL36-inspired)
type: expression
flag: rule_g_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Supply_Air_Temperature_Setpoint:
    brick: Supply_Air_Temperature_Setpoint
  Damper_Position_Command:
    brick: Damper_Position_Command
  Valve_Command:
    brick: Valve_Command

params:
  oat_tol: 1.15
  fan_delta_t: 0.55
  sat_tol: 1.15
  econ_min_open: 0.12

```

```

expression: |
    (Outside_Air_Temperature_Sensor - oat_tol >
    Supply_Air_Temperature_Setpoint - fan_delta_t + sat_tol) &
    (Damper_Position_Command > econ_min_open) & (Valve_Command < 0.1)

```

18.5.8 Rule H — Ambient vs blended mismatch (econ + mech cooling)

When both economizer and mechanical cooling are active, MAT should approach OAT. Large deviation suggests inadequate mixing or damper fault. *Adapted from GL36 AFDD guidance.*

```

name: ambient_blend_mismatch_econ_mech
description: Outdoor and blended air diverge in econ+mech cooling
(GL36-inspired)
type: expression
flag: rule_h_flag
equipment_type: [AHU, VAV_AHU]

inputs:
    Outside_Air_Temperature_Sensor:
        brick: Outside_Air_Temperature_Sensor
    Mixed_Air_Temperature_Sensor:
        brick: Mixed_Air_Temperature_Sensor
    Valve_Command:
        brick: Valve_Command
    Damper_Position_Command:
        brick: Damper_Position_Command

params:
    oat_tol: 1.15
    blend_tol: 1.15

expression: |
    (np.abs(Mixed_Air_Temperature_Sensor -
    Outside_Air_Temperature_Sensor) > np.sqrt(blend_tol**2 +
    oat_tol**2)) & (Valve_Command > 0.01) & (Damper_Position_Command >
    0.9)

```

18.5.9 Rule I — Ambient vs blended mismatch (econ-only)

In economizer-only mode, MAT should match OAT. Deviation indicates damper or mixing fault. *Adapted from GL36 AFDD guidance.*

```

name: ambient_blend_mismatch_econ
description: Outdoor and blended air diverge in economizer-only
mode (GL36-inspired)

```

```

type: expression
flag: rule_i_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Mixed_Air_Temperature_Sensor:
    brick: Mixed_Air_Temperature_Sensor
  Damper_Position_Command:
    brick: Damper_Position_Command

params:
  oat_tol: 1.15
  blend_tol: 1.15

expression: |
  (np.abs(Mixed_Air_Temperature_Sensor -
  Outside_Air_Temperature_Sensor) > np.sqrt(blend_tol**2 +
  oat_tol**2)) & (Damper_Position_Command > 0.9)

```

18.5.10 Rule J — Discharge above blended in cooling

SAT exceeds MAT when cooling (econ+mech or mech-only) is active. Indicates underperforming cooling coil or valve. *Adapted from GL36 AFDD guidance.*

```

name: discharge_above_blend_cooling
description: Discharge air above blended air in cooling modes
(GL36-inspired)
type: expression
flag: rule_j_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor
  Mixed_Air_Temperature_Sensor:
    brick: Mixed_Air_Temperature_Sensor
  Valve_Command:
    brick: Valve_Command
  Damper_Position_Command:
    brick: Damper_Position_Command

params:
  fan_delta_t: 0.55
  blend_tol: 1.15
  sat_tol: 1.15
  econ_min_open: 0.12

```

```

        expression: |
            (Supply_Air_Temperature_Sensor > Mixed_Air_Temperature_Sensor +
np.sqrt(sat_tol**2 + blend_tol**2) + fan_delta_t) &
            (((Damper_Position_Command > 0.9) & (Valve_Command > 0)) |
            ((Damper_Position_Command <= econ_min_open) & (Valve_Command >
0.9)))

```

18.5.11 Rule K — Discharge above setpoint in full cooling

SAT above setpoint with cooling at full capacity. Suggests undersized coil or plant limits. *Adapted from GL36 AFDD guidance.*

```

        name: discharge_above_sp_full_cool
        description: Discharge air above setpoint in full cooling mode
(GL36-inspired)
        type: expression
        flag: rule_k_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
            Supply_Air_Temperature_Sensor:
                brick: Supply_Air_Temperature_Sensor
            Supply_Air_Temperature_Setpoint:
                brick: Supply_Air_Temperature_Setpoint
            Valve_Command:
                brick: Valve_Command
            Damper_Position_Command:
                brick: Damper_Position_Command

        params:
            sat_tol: 1.15
            econ_min_open: 0.12

        expression: |
            (Supply_Air_Temperature_Sensor > Supply_Air_Temperature_Setpoint
+ sat_tol) & (((Damper_Position_Command > 0.9) & (Valve_Command >
0.9)) | ((Damper_Position_Command <= econ_min_open) &
(Valve_Command > 0.9)))

```

18.5.12 Rule L — Cooling coil delta-T when inactive

Temperature drop across cooling coil when it should be off. Indicates leaking CHW valve or coil bypass. *Adapted from GL36 AFDD guidance.*

```

        name: clg_coil_drop_when_off
        description: Temperature drop across cooling coil when it should
be off (GL36-inspired)
        type: expression

```

```

flag: rule_l_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Cooling_Coil_Entering_Air_Temperature_Sensor:
    brick: Cooling_Coil_Entering_Air_Temperature_Sensor
  Cooling_Coil_Leaving_Air_Temperature_Sensor:
    brick: Cooling_Coil_Leaving_Air_Temperature_Sensor
  Heating_Valve_Command:
    brick: Valve_Command
  Cooling_Valve_Command:
    brick: Valve_Command
  Damper_Position_Command:
    brick: Damper_Position_Command

params:
  enter_tol: 1.15
  leave_tol: 1.15
  econ_min_open: 0.12

expression: |
  ((Cooling_Coil_Entering_Air_Temperature_Sensor -
Cooling_Coil_Leaving_Air_Temperature_Sensor) >
np.sqrt(enter_tol**2 + leave_tol**2)) & (((Heating_Valve_Command >
0) & (Cooling_Valve_Command == 0) & (Damper_Position_Command <=
econ_min_open)) | ((Heating_Valve_Command == 0) &
(Cooling_Valve_Command == 0) & (Damper_Position_Command >
econ_min_open)))

```

18.5.13 Rule M — Heating coil delta-T when inactive

Temperature rise across heating coil when it should be off.
Indicates leaking HW valve. *Adapted from GL36 AFDD guidance.*

```

name: htg_coil_rise_when_off
description: Temperature rise across heating coil when it should
be off (GL36-inspired)
type: expression
flag: rule_m_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Heating_Coil_Entering_Air_Temperature_Sensor:
    brick: Heating_Coil_Entering_Air_Temperature_Sensor
  Heating_Coil_Leaving_Air_Temperature_Sensor:
    brick: Heating_Coil_Leaving_Air_Temperature_Sensor
  Heating_Valve_Command:
    brick: Valve_Command
  Cooling_Valve_Command:
    brick: Valve_Command

```

```

    Damper_Position_Command:
        brick: Damper_Position_Command

    params:
        enter_tol: 1.15
        leave_tol: 1.15
        fan_delta_t: 0.55
        econ_min_open: 0.12

    expression: |
        ((Heating_Coil_Leaving_Air_Temperature_Sensor -
Heating_Coil_Entering_Air_Temperature_Sensor) >
np.sqrt(enter_tol**2 + leave_tol**2) + fan_delta_t) &
(((Heating_Valve_Command == 0) & (Cooling_Valve_Command == 0) &
(Damper_Position_Command > econ_min_open)) |
((Heating_Valve_Command == 0) & (Cooling_Valve_Command > 0) &
(Damper_Position_Command > 0.9)) | ((Heating_Valve_Command == 0) &
(Cooling_Valve_Command > 0) & (Damper_Position_Command <=
econ_min_open)))

```

Heat exchanger effectiveness — see [rule types \(erv_efficiency\)](#).

18.6 Central plant

18.6.1 Differential pressure at max pump speed

Variable-speed pump cannot meet differential pressure setpoint at full speed. Indicates piping issues, undersized pump, or blocked strainers.

```

    name: dp_below_sp_pump_max
    description: Differential pressure below setpoint with pump at
full speed
    type: expression
    flag: dp_pump_flag

    inputs:
        Differential_Pressure_Sensor:
            brick: Differential_Pressure_Sensor
        Differential_Pressure_Setpoint:
            brick: Differential_Pressure_Setpoint
        Pump_Speed_Command:
            brick: Pump_Speed_Command

    params:
        dp_margin: 2.2
        pmp_hi_frac: 0.93
        pmp_near_hi: 0.06

```

```
expression: |
    (Differential_Pressure_Sensor < Differential_Pressure_Setpoint -
dp_margin) & (Pump_Speed_Command >= pmp_hi_frac - pmp_near_hi)
```

18.6.2 Plant flow high at max pump

Flow unusually high with pump at high speed. Suggests short circuit or flow meter error.

```
name: flow_high_pump_max
description: Water flow unusually high with pump at full speed
type: expression
flag: flow_high_flag

inputs:
  Water_Flow_Sensor:
    brick: Water_Flow_Sensor
  Pump_Speed_Command:
    brick: Pump_Speed_Command

params:
  flow_hi_limit: 1100.0
  pmp_hi_frac: 0.93
  pmp_near_hi: 0.06

expression: |
    (Water_Flow_Sensor > flow_hi_limit) & (Pump_Speed_Command >=
pmp_hi_frac - pmp_near_hi)
```

18.6.3 Plant supply temp outside deadband

CHW supply temperature outside deadband while pump runs. Requires coil entering/leaving or plant supply temp sensors.

```
name: plant_supply_temp_deadband
description: Supply water temp outside deadband during pump
operation
type: expression
flag: chw_temp_fault

inputs:
  Chilled_Water_Supply_Temperature_Sensor:
    brick: Chilled_Water_Supply_Temperature_Sensor
  Chilled_Water_Supply_Temperature_Setpoint:
    brick: Chilled_Water_Supply_Temperature_Setpoint
  Pump_Speed_Command:
    brick: Pump_Speed_Command

params:
  sp_band: 2.2
```

```

expression: |
    (Pump_Speed_Command > 0.01) &
    ((Chilled_Water_Supply_Temperature_Sensor <
    Chilled_Water_Supply_Temperature_Setpoint - sp_band) |
    (Chilled_Water_Supply_Temperature_Sensor >
    Chilled_Water_Supply_Temperature_Setpoint + sp_band))

```

18.6.4 Chiller runtime over daily limit

Chiller running beyond a daily threshold (e.g. 23 hours). Often indicates over-cooling or schedules that bypass lockout. Use a rolling window sized for your data interval (e.g. 5-min data → 276 samples ≈ 23 h).

```

name: chiller_excessive_runtime
description: Chiller runtime exceeds daily threshold (rolling
window)
type: expression
flag: chiller_runtime_fault

inputs:
    Chiller_Status:
        brick: Chiller_Status

params:
    # For 5-min data: 23 hours ≈ 276 samples; max_runtime = count of
    "on" samples in window
    rolling_samples: 276
    max_runtime_samples: 264

expression: |
    Chiller_Status.rolling(window=rolling_samples).sum() >
max_runtime_samples

```

Note: Adjust rolling_samples and max_runtime_samples for your data interval. For 5-min data, 264 samples ≈ 22 hours.

18.7 Heat pumps

18.7.1 Discharge cold when heating

Flags when discharge air temperature is below a minimum (e.g. 80°F) while the supply fan is running. Indicates the heat pump is not heating effectively—possible issues with compressor, refrigerant, or reversing valve. Tunable via min_discharge_temp. Logic: if zone temp < 69°F (heating mode), the discharge should be warm; a cold discharge with the fan on and a cold zone indicates the heat pump is failing to heat.

```

    name: hp_discharge_cold_when_heating
    description: Discharge air below minimum when fan on and zone is
cold (heating mode)
    type: expression
    flag: hp_discharge_cold_flag
    equipment_type: [Heat_Pump]

    inputs:
      Supply_Air_Temperature_Sensor:
        brick: Supply_Air_Temperature_Sensor
      Zone_Temperature_Sensor:
        brick: Zone_Temperature_Sensor
      Supply_Fan_Status:
        brick: Supply_Fan_Status

    params:
      min_discharge_temp: 85
      zone_cold_threshold: 69.0
      fan_on_threshold: 0.01

    expression: |
      (Supply_Fan_Status > fan_on_threshold) &
      (Zone_Temperature_Sensor < zone_cold_threshold) &
      (Supply_Air_Temperature_Sensor < min_discharge_temp)

```

18.8 VAV zones

18.9 Standard AHU/VAV starter pack (practical baseline)

These starter rules map directly to the provided YAML templates and are good defaults for initial deployments:

- 01_vav_zone_temp_bounds_occupied.yaml
- 02_vav_zone_temp_flatline_occupied.yaml
- 03_vav_damper_command_extreme_flatline.yaml
- 04_ahu_runtime_outside_schedule.yaml
- 05_ahu_duct_static_pressure_not_maintained.yaml
- 06_ahu_internal_temp_sensor_bounds.yaml
- 07_ahu_internal_temp_sensor_flatline.yaml

Use these as a baseline, then tune:

- **Schedules:** occupied_start_hour, occupied_end_hour
- **Rolling persistence:** rolling_window, roll_samples
- **Deadbands/tolerances:** *_tolerance_*, *_deadband_*

- **Command scaling:** use `normalize_cmd(...)` when command units may be 0-100 instead of 0-1

18.9.1 Starter: economizer 100% OA temperature tracking

When OA damper is effectively fully open and fan is on, mixed/leaving temperatures should closely track outdoor air (subject to sensor + fan heat tolerance).

```

name: ahu_econ_100oa_temp_tracking_fault
description: At (near) 100% OA, MAT and SAT should track OAT
within tolerance
type: expression
flag: ahu_econ_100oa_temp_tracking_fault
equipment_type: [AHU, VAV_AHU]

inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Mixed_Air_Temperature_Sensor:
    brick: Mixed_Air_Temperature_Sensor
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor
  Damper_Position_Command:
    brick: Damper_Position_Command
  Supply_Fan_Status:
    brick: Supply_Fan_Status

params:
  # OA damper threshold in fraction (0.95 = 95% open)
  damper_near_full_open: 0.95
  fan_on_threshold: 0.5
  # Imperial: 2.0 F ~= 1.1 C
  temp_tracking_tol_f: 2.0
  # Optional fan heat allowance; Imperial: 0.5 F ~= 0.3 C
  fan_heat_allowance_f: 0.5

expression: |
  (Supply_Fan_Status > fan_on_threshold)
  & (normalize_cmd(Damper_Position_Command) >=
damper_near_full_open)
  & (
    (np.abs(Mixed_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor) > temp_tracking_tol_f)
    | (np.abs((Supply_Air_Temperature_Sensor -
fan_heat_allowance_f) - Mixed_Air_Temperature_Sensor) >
temp_tracking_tol_f)
  )

```

18.9.2 Starter: mechanical cooling when free cooling should be used

Flags when cooling valve is active while outdoor conditions are favorable for economizer cooling.

```
name: ahu_mech_cooling_when_free_cooling_available
description: Mechanical cooling active when OAT is low enough for
free cooling opportunity
type: expression
flag: ahu_mech_cooling_when_free_cooling_available_fault
equipment_type: [AHU, VAV_AHU]

inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Supply_Air_Temperature_Setpoint:
    brick: Supply_Air_Temperature_Setpoint
  Damper_Position_Command:
    brick: Damper_Position_Command
  Valve_Command:
    brick: Valve_Command

params:
  # Imperial defaults; equivalent Celsius values in comments
  free_cooling_margin_f: 2.0      # ~= 1.1 C
  damper_low_open_max: 0.50      # 50% open
  cooling_cmd_on_min: 0.10

expression: |
  (Outside_Air_Temperature_Sensor <=
Supply_Air_Temperature_Setpoint - free_cooling_margin_f)
  & (normalize_cmd(Damper_Position_Command) < damper_low_open_max)
  & (normalize_cmd(Valve_Command) >= cooling_cmd_on_min)
```

18.9.3 Starter: OA damper too open in hot (OAT > 70 F / 21.1 C) or cold (OAT < 40 F / 4.4 C) ambient

This captures two common waste/comfort risk conditions:

- **Hot ambient:** damper 50%+ open when OAT > 70 F (21.1 C)
- **Very cold ambient:** damper 50%+ open when OAT < 40 F (4.4 C)

```
name: ahu_oa_damper_excess_open_extreme_ambient
description: OA damper too open under hot or very cold outdoor
conditions
type: expression
flag: ahu_oa_damper_excess_open_extreme_ambient_fault
equipment_type: [AHU, VAV_AHU]
```

```

inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Damper_Position_Command:
    brick: Damper_Position_Command

params:
  oa_damper_open_min: 0.50
  high_oat_threshold_f: 70.0    # 21.1 C
  low_oat_threshold_f: 40.0    # 4.4 C

expression: |
  (normalize_cmd(Damper_Position_Command) >= oa_damper_open_min)
  & (
    (Outside_Air_Temperature_Sensor >= high_oat_threshold_f)
    | (Outside_Air_Temperature_Sensor <= low_oat_threshold_f)
  )

```

18.9.4 Metric equivalents quick note

If your site trends are in Celsius, convert thresholds accordingly:

- 69 F -> 20.6 C
- 75 F -> 23.9 C
- 70 F -> 21.1 C
- 63 F -> 17.2 C
- 40 F -> 4.4 C
- 35 F -> 1.7 C
- 120 F -> 48.9 C
- 2.0 F tolerance -> 1.1 C
- 0.5 F fan allowance -> 0.3 C

For SI pressure, 0.15 inH2O is about 37 Pa.

18.9.5 Excessive heating during warm weather

Reheat valve open when outdoor air is warm. Suggests over-cooling or setpoint issues.

```

name: zone_reheat_warm_ambient
description: Heating valve open when OAT is high
type: expression
flag: excessive_heating_flag
equipment_type: [VAV]

inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Reheat_Valve_Command:
    brick: Valve_Command

```

```

params:
  t_amb_cutoff: 78.0
  reheat_open_min: 0.52

expression: |
  (Outside_Air_Temperature_Sensor > t_amb_cutoff) &
  (Reheat_Valve_Command > reheat_open_min)

```

18.9.6 Damper or valve at full open

Damper or reheat valve consistently at full open. Indicates override, undersized equipment, or control fault.

```

name: zone_damper_valve_full_open
description: Damper or valve at full open for extended period
type: expression
flag: damper_100_flag
equipment_type: [VAV]

inputs:
  Damper_Position_Command:
    brick: Damper_Position_Command

params:
  full_open_pct: 97.5
  roll_samples: 105

expression: |
  (Damper_Position_Command > full_open_pct) &
  (Damper_Position_Command.rolling(roll_samples).min() >
  full_open_pct)

```

18.9.7 Zone and IAQ bounds

For CO2 and zone temperature out-of-range checks, use the bounds rule type — `co2_bounds` and `zone_temp_bounds` examples.

18.10 Opportunistic rules (economizer & ventilation)

18.10.1 Economizing when outdoor conditions are unfavorable

Economizer active when outdoor air is too warm or humid. Use OAT (or enthalpy if available) vs. threshold.

```

    name: econ_active_warm_ambient
    description: OA damper open when outdoor conditions do not favor
economizing
    type: expression
    flag: econ_when_shouldnt_flag
    equipment_type: [AHU, VAV_AHU]

    inputs:
      Outside_Air_Temperature_Sensor:
        brick: Outside_Air_Temperature_Sensor
      Damper_Position_Command:
        brick: Damper_Position_Command

    params:
      t_amb_econ_cutoff: 63.0
      dpr_econ_min: 0.42

    expression: |
      (Outside_Air_Temperature_Sensor > t_amb_econ_cutoff) &
(Damper_Position_Command > dpr_econ_min)

```

18.10.2 Mechanical cooling when econ could suffice

Cooling valve open when outdoor air is cool enough for economizer. Opportunity to reduce mechanical cooling.

```

    name: mech_cool_when_econ_available
    description: Mechanical cooling active when economizing could
suffice
    type: expression
    flag: cooling_when_econ_flag
    equipment_type: [AHU, VAV_AHU]

    inputs:
      Outside_Air_Temperature_Sensor:
        brick: Outside_Air_Temperature_Sensor
      Damper_Position_Command:
        brick: Damper_Position_Command
      Valve_Command:
        brick: Valve_Command

    params:
      t_amb_econ_cutoff: 63.0
      dpr_not_econ_max: 0.32

    expression: |
      (Outside_Air_Temperature_Sensor < t_amb_econ_cutoff) &
(Damper_Position_Command < dpr_not_econ_max) & (Valve_Command >
0.01)

```

18.10.3 Low ventilation (estimated OA fraction)

For units without airflow meters, estimate OA fraction from OAT, MAT, RAT. Flag when below minimum design OA.

```
name: low_oa_fraction_estimated
description: Estimated OA fraction below minimum (OAT, MAT, RAT
method)
type: expression
flag: low_vent_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Mixed_Air_Temperature_Sensor:
    brick: Mixed_Air_Temperature_Sensor
  Return_Air_Temperature_Sensor:
    brick: Return_Air_Temperature_Sensor
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Supply_Fan_Speed_Command:
    brick: Supply_Fan_Speed_Command

params:
  oa_min_pct: 21.0
  t_rat_oat_min_gap: 2.2

expression: |
  (Supply_Fan_Speed_Command > 0.01) &
  (np.abs(Return_Air_Temperature_Sensor -
  Outside_Air_Temperature_Sensor) > t_rat_oat_min_gap) &
  (((Mixed_Air_Temperature_Sensor - Return_Air_Temperature_Sensor) /
  (Outside_Air_Temperature_Sensor - Return_Air_Temperature_Sensor) *
  100) < oa_min_pct)
```

Note: Guard against division-by-zero when outdoor and return temps are close.

18.10.4 Preheat over-conditioning

Preheat coil leaving temp higher than needed (e.g. above OAT when OAT > SAT SP, or above SAT SP when OAT < SAT SP). Indicates wasted heating energy.

```
name: preheat_excess_temp
description: Preheat coil leaving temp above required level
type: expression
flag: preheat_waste_flag
equipment_type: [AHU, VAV_AHU]

inputs:
  Preheat_Coil_Leaving_Air_Temperature_Sensor:
```

```

    brick: Preheat_Coil_Leaving_Air_Temperature_Sensor
Supply_Air_Temperature_Setpoint:
    brick: Supply_Air_Temperature_Setpoint
Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
Valve_Command:
    brick: Valve_Command

params:
    excess_tol: 2.2

expression: |
    (Valve_Command > 0.01) & (((Outside_Air_Temperature_Sensor >
Supply_Air_Temperature_Setpoint) &
    (Preheat_Coil_Leaving_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor > excess_tol)) |
    ((Outside_Air_Temperature_Sensor <
Supply_Air_Temperature_Setpoint) &
    (Preheat_Coil_Leaving_Air_Temperature_Sensor -
Supply_Air_Temperature_Setpoint > excess_tol)))

```

18.10.5 Blended air damper deviation

Expected MAT (from OAT, RAT, damper positions) differs from measured MAT. Indicates damper leakage or faulty mixing.

Requires airflow or damper position data; logic is more involved. A simplified version compares MAT to a weighted blend of OAT and RAT when dampers suggest significant OA.

18.11 Weather station

weather_temp_stuck (flatline) — see [flatline rule type](#).

18.11.1 Unrealistic temperature spike

Temperature change between consecutive readings exceeds physical limit.

```

name: weather_temp_spike
description: Unrealistic temperature change between readings
type: expression
flag: fault_temp_spike

inputs:
    Outside_Air_Temperature_Sensor:
        brick: Outside_Air_Temperature_Sensor

```

```

params:
  spike_limit: 16.0

expression: |
  Outside_Air_Temperature_Sensor.diff().abs() > spike_limit

```

18.11.2 RH bounds

For relative humidity out-of-range, use the bounds rule type — rh_bounds example.

18.11.3 Wind gust vs sustained

```

name: weather_gust_lt_wind
description: Wind gust reported lower than sustained wind
type: expression
flag: fault_gust_lt_wind

inputs:
  Wind_Gust_Speed_Sensor:
    brick: Wind_Gust_Speed_Sensor
  Wind_Speed_Sensor:
    brick: Wind_Speed_Sensor

expression: |
  Wind_Gust_Speed_Sensor.notna() & Wind_Speed_Sensor.notna() &
  (Wind_Gust_Speed_Sensor < Wind_Speed_Sensor)

```

18.12 Sensor validation (bounds & flatline)

Use the bounds and flatline rule types for generic sensor checks. Typical bounds:

Sensor type	Min	Max
Zone temp	40	100
Supply air temp	40	150
Air pressure (inH2O)	-5	10
RH	0	100
Chilled water temp	35	100
Hot water temp	50	212
Condenser water	50	110
CO2 (ppm)	400	2000

18.13 Follow-up (RFCs): multi-ontology selectors & cookbook matrix

Multi-ontology inputs today use the per-input fields **brick**, **haystack**, **dbo**, **s223**, **223p** (see [Ontology labels](#)) plus **column_map**. Deeper graph and API workflows live in **[open-fdd-afdd-stack](#)**.

Cookbook matrix / generator: When a selector schema is stable, we can add a **generated appendix** (logical input × Brick × Haystack × DBO × 223P notes) from a single CSV/YAML source so the long cookbook does not drift. Until then, see the **open-fdd** repo's [examples/column_map_resolver_workshop/simple_ontology_demo.py](#) (with [simple_ontology_rule.yaml](#)).

Next: [Verification](#) · [Column map resolvers](#)

19 Test bench rule catalog

20 Rule YAML catalog (in this repository)

Example and test rules live next to the engine sources. Use them as templates for your own `rules_dir`.

20.1 `open_fdd/tests/fixtures/rules/` (pytest)

File	Purpose
<code>weather_gust_lt_wind.yaml</code>	Weather / wind gust check
<code>weather_rh_out_of_range.yaml</code>	RH bounds
<code>weather_temp_spike.yaml</code>	Temperature spike
<code>weather_temp_stuck.yaml</code>	Temperature stuck
<code>test_flatline_sat.yaml</code>	Flatline SAT
<code>test_sensor_flatline_1774618711.yaml</code>	Sensor flatline test

20.2 examples/AHU/rules/ (notebooks / demos)

File	Purpose
sensor_bounds.yaml	Bounds example
sensor_flatline.yaml	Flatline example
sat_operating_band.yaml	SAT band

20.3 Related docs

- [Fault rules overview](#)
- [Expression rule cookbook](#)

21 YAML rules → Pandas (under the hood)

22 YAML rules → Pandas DataFrames (under the hood)

This note is for engineers who want to see **exactly** how Open-FDD turns **rule YAML on disk** into **pandas operations** on **time-series DataFrames**—and where Brick TTL fits in. It complements [Fault rules overview](#) and [standalone CSV / pandas](#).

22.1 1. YAML never becomes “one giant DataFrame of rules”

Rule files are **configuration**, not tabular data. Open-FDD:

1. Reads each *.yaml in rules_dir with PyYAML (yaml.safe_load) into a **Python dict** per file (open_fdd.engine.runner.load_rule / load_rules_from_dir).
2. Keeps those dicts in a **list[dict]** inside RuleRunner (RuleRunner(rules=...) or RuleRunner(rules_path=...)).
3. At evaluation time, walks that list and, for each rule, computes a **boolean mask** (pandas.Series) aligned to the **sensor**

DataFrame's index, then writes a **new column** for the fault flag (e.g. `my_rule_flag`).

So: **rules = list of dicts in memory; data = one wide-ish DataFrame** of timestamps and point columns.

22.2 2. From long telemetry rows to a wide DataFrame

A common pattern (warehouse export, historian CSV, SQL query) is a **long** table (timestamp, point_id, value). For **RuleRunner** you typically:

1. Load rows into pandas.
2. **Pivot to wide**: `df.pivot_table(index="timestamp", columns="point_id", values="value")` (or equivalent) so each sensor is a column.
3. **Rename columns** using `column_map` (dict or manifest) so rule inputs line up with Brick-style or logical keys.
4. Parse the index with `pd.to_datetime` when you need time-based checks.

pandas handles aggregation during the pivot when duplicate index/column pairs exist; the result is one row per timestamp.

22.3 3. What RuleRunner.run does to that DataFrame

`RuleRunner.run(open_fdd.engine.runner):`

1. `result = df.copy()` — rules never mutate the caller's frame in place (callers can still hold a reference to the original).
2. For **each** rule dict:
 - Derives `flag_name` from `flag` or `{name}_flag`.
 - Calls `_evaluate_rule` → returns a **boolean Series** (fault mask) aligned to `result.index`.
 - Optionally applies a **rolling window** on the mask:
`mask.astype(int).rolling(window=rw).sum() >= rw` so a fault must persist for N consecutive samples.
 - Assigns `result[flag_name] = ...` as integer 0/1.

So each rule adds **one column** of flags; the frame grows **width-wise**, not row-wise.

22.4 4. How a YAML rule dict becomes pandas expressions

Inside `_evaluate_rule`, Open-FDD branches on `rule["type"]` (e.g. `bounds`, `flatline`, `expression`, ...):

- **column_map resolution:** For each logical input key, the runner picks a **DataFrame column name**. If the YAML input has a **brick** class, the supplied **column_map** dict is consulted (brick class key → column label), so the **same YAML** can run against different exports as long as the map matches your frame.
- **Bounds / thresholds:** Typical pattern is `(series < low) | (series > high)` using **vectorized** comparisons — these are **numpy ufuncs** under pandas, no Python for over rows.
- **Expressions:** String expressions may be evaluated in a **restricted eval** context (`open_fdd.engine.checks.check_expression`) with named series bound to `result[col]` — still vectorized.

If a required column is missing, the runner either **raises** or **skips** the rule (`skip_missing_columns=True`), depending on the call site.

22.5 5. Reloading rules vs DataFrame lifetime

If you call `load_rules_from_dir` before each **RuleRunner** construction, edits to YAML on disk take effect on the **next** run. The **DataFrame** you pass in exists only for that evaluation; construct a fresh frame for each batch or window as your pipeline requires.

22.6 6. Mental model checklist

Artifact	In-memory shape	pandas role
Rule YAML files	N/A (on disk)	None until loaded
Loaded rules	<code>list[dict]</code>	None
Telemetry window	<code>DataFrame</code> (time × points)	pivot, datetime index, column rename
Rule output	Same index, +flag columns	vectorized masks, optional rolling
Downstream store	Your app persists outputs if needed	After flags are computed

For a **minimal** example of rules + CSV (no DB), see [standalone CSV / pandas](#) and unit tests in `open_fdd/tests/engine/test_runner.py`.

23 Fault rules for HVAC

24 Fault rules for HVAC

FDD fault rule types, YAML format, and expression cookbook for HVAC and building systems. Rules are **100% Brick-model driven**: inputs reference Brick classes only; column mapping comes from the Brick TTL and [Brick external timeseries references](#) ([timeseries storage](#)), which Open-FDD embraces at the heart of FDD.

Page	Description
Overview	Rule types, RuleRunner, column maps, hot reload
Test bench rule catalog	Links to example YAML under <code>open_fdd/tests/fixtures/rules/</code> and <code>examples/AHU/rules/</code>

25 Security

26 Security

The `open-fdd` PyPI package is a **Python library** for evaluating rules on in-memory (or local) **pandas** data. It does **not** open network ports or ship a hosted service.

26.1 Supply chain and dependencies

- **Pinned versions** — follow your organization's policy for pinning `open-fdd` and its dependencies (**pandas**, **numpy**, **pyyaml**, **pydantic**) in applications.
 - **Verify PyPI artifacts** — use `twine check` and hashes in lockfiles where applicable.
-

26.2 Rule YAML and expressions

- **Trust model** — only load rule YAML from **trusted** paths (version-controlled configs, sealed containers, or signed bundles). Treat rule files like code.
 - **Expression rules** — use the documented expression surface; avoid piping untrusted strings directly into rule files without review.
-

26.3 Reporting issues

Report security-sensitive bugs through the repository's **Security policy** (if enabled) or maintainer contact on the project README.

27 Cloud export example

28 Cloud export example

Open-FDD runs **inside the building, behind the firewall**. It does not push data to the cloud itself. Cloud-based FDD providers, MSI (Monitoring, Savings, and Intelligence) vendors, and commissioning or IoT contractors run their **own** export process: a small script or edge gateway on the building or OT network that **pulls** from the Open-FDD API over the LAN and then sends that data to their cloud or analytics platform. See [Behind the firewall: cloud export is vendor-led](#) on the docs home.

This page describes the **cloud export example** in the repo: a minimal Python script that demonstrates how vendor X, Y, or Z could use the Open-FDD API to get fault and timeseries data and use it as a starting point for their cloud pipeline.

28.1 What the example does

The script `examples/cloud_export.py` calls standard Open-FDD download and analytics endpoints and prints the results. In a real integration you would replace the print steps with your own logic: POST to your REST API, write to S3, push to Azure IoT Hub or AWS IoT, send to SkySpark or another analytics platform, etc.

Step	Endpoint	Purpose
1	GET /download/faults?format=json	Fault results for programmatic ingestion (JSON)
2	GET /download/faults?format=csv	Fault CSV (Excel-friendly)
3	GET /analytics/motor-runtime	Motor runtime (data-model driven)
4	GET /download/csv	Timeseries wide-format CSV
5	GET /analytics/fault-summary	Fault counts by fault_id

All requests use date range and optional site_id; the API docs at / docs list full parameters.

28.2 Run the example

Prerequisites: Open-FDD API running (e.g. `http://localhost:8000`), pip install `httpx`.

```
# From repo root
python examples/cloud_export.py
python examples/cloud_export.py --site default --days 14
API_BASE=http://your-openfdd-host:8000 python examples/
cloud_export.py
```

Docker: See [examples/cloud_export/README.md](#) for building and running the example in a container (e.g. on a vendor edge device that reaches the Open-FDD API over the LAN).

28.3 Use it as your starting point

1. **Clone or copy** `examples/cloud_export.py` into your integration codebase.
2. **Keep** the HTTP calls to `/download/faults`, `/download/csv`, and the analytics endpoints; adjust `API_BASE` (or `env`) to point at the customer's Open-FDD instance.
3. **Replace** the `print()` / local handling with your cloud send: REST POST, message queue, blob storage, or IoT hub. Add auth (e.g. API key or TLS client cert) if you expose the puller to a wider network.

4. **Schedule** the script (cron, systemd timer, or container restart policy) so you poll the Open-FDD API at the interval that fits your product (e.g. hourly or daily fault sync).

Open-FDD does not initiate outbound cloud connections or manage your data transmission; the example shows how your process can pull data from Open-FDD and then send it to your cloud for deeper insights.

29 RDF, Brick, and optional BACnet-shaped metadata

30 RDF, Brick, and optional BACnet-shaped metadata

The `open-fdd` wheel evaluates rules on `pandas` using `column_map` (dict or manifest). **Brick**, **223P**, **Haystack**, **DBO**, and BACnet-shaped **TTL** are **not** parsed inside this package—you build the bridge in your pipeline or use a platform that does.

Where the full graph story lives: [open-fdd-afdd-stack — docs/](#) (CRUD, SPARQL, import/export, compose, hosted services).

Here in this repo: see [Column map resolvers](#) and the [Expression rule cookbook](#) for ontology-agnostic rule authoring.

31 Context and recordkeeping

32 Context and recordkeeping

Open-FDD context should be durable, visible, and reusable by both humans and agents.

32.1 Rule

If context affects repeatable operations, store it in versioned docs instead of leaving it in chat-only memory.

32.2 Good context to commit

- Verification strategy and sweep logic.
- BACnet/model assumptions that affect diagnostics.
- Operator playbook updates and recurring workflow improvements.
- Cross-check guidance for frontend/API parity and model-based telemetry.

32.3 Context anti-patterns

- One-machine-only tribal notes with no repo trace.
- Secrets in Markdown.
- Raw transcript dumps without distillation.

33 Operational states

34 Operational states

Open-FDD operations should explicitly identify environment mode before asserting correctness.

34.1 States

- TEST_BENCH: deterministic fake-device conditions or lab-only setup.
- LIVE_HVAC: production building telemetry and real-world operating behavior.

34.2 Why this matters

The same observed value can be expected in bench mode and suspicious in live mode. Mode-aware interpretation avoids false positives and noisy operations.

35 Concepts

36 Concepts

Background for how **open-fdd** fits into analytics workflows. **Sites, equipment, SPARQL, and HTTP APIs** are covered in **open-fdd-afdd-stack — docs/**.

Page	Description
RDF, Brick, and optional BACnet-shaped metadata	TTL/graph patterns <i>outside</i> the PyPI wheel; column_map bridges into pandas.
Cloud export example	Pattern for exporting fault summaries from your own services.
Context and recordkeeping	What belongs in versioned docs vs local notes.
Operational states	Test bench vs live HVAC interpretation.

See also the [Documentation home](#) and [Column map resolvers](#).

37 How-to Guides

38 How-to Guides

Recipes for `pip install open-fdd`, releases, and embedding RuleRunner on pandas.

Full platform (data model, APIs, compose, dashboards): **open-fdd-afdd-stack — docs/**

38.1 Engine

- [PyPI releases \(open-fdd\)](#) — tags, trusted publishing, twine check.
- [The optional openfdd-engine package](#) — openfdd_engine vs open_fdd.engine.
- [Engine-only deployment and external IoT pipelines](#) — RuleRunner on DataFrames.

- Desktop app (under construction) — local ingest, Feather storage, BRICK sync, and batched rule runs.
- Verification — pytest and small smoke checks.
- Cloning and porting — portable rules and envs.
- Operations (engine) — CI and scheduling notes.
- Danger zone — expression safety and sharp edges.
- Quick reference — imports and common commands.

Published site: bbartling.github.io/open-fdd

39 PyPI releases (open-fdd)

40 PyPI releases (open-fdd)

Canonical public install for the 2.x rules engine:

```
pip install open-fdd
```

That distribution ships `open_fdd.engine` (RuleRunner, YAML rules, same stack as the platform FDD loop). **Why does PyPI “Release history” mix 0.1.x and 2.x?** That is still **one** project, `open-fdd`: older **0.1.x** uploads were the legacy, hard-coded pandas era; **2.x** is the current config-driven engine + platform-capable tree. Pip resolves the latest **2.x** unless you pin an old version.

40.1 openfdd-engine (optional second PyPI name)

The repo also contains `packages/openfdd-engine/` (import name `openfdd_engine`): a thin re-export that **depends on open-fdd**. Its **version is kept the same as open-fdd 2.x** (e.g. both 2.0.13) so release numbers stay easy to reason about, even though it is a **separate** PyPI project name.

Publishing uses `.github/workflows/publish-openfdd-engine.yml`: push tag `openfdd-engine-vX.Y.Z` after the `openfdd-engine` PyPI project has a **trusted publisher** for that workflow (same OIDC idea as `open-fdd`). `workflow_dispatch` builds only; upload runs on matching tags.

40.2 Release baseline (PyPI)

- Check live version:

```
curl -s https://pypi.org/pypi/open-fdd/json | python3 -c "import sys, json; print(json.load(sys.stdin)['info']['version'])"
```

40.3 1) Before a release of open-fdd

1. **Version** — Root `pyproject.toml` → [project] version (e.g. 2.0.11).
2. **openfdd-engine** — Bump `packages/openfdd-engine/pyproject.toml` version to the **same** `X.Y.Z` as `open-fdd`, and set `open-fdd>=X.Y.Z` in dependencies. Tag `openfdd-engine-vX.Y.Z` if you publish that project.

40.3.1 Transition checklist (legacy 0.1.x → 2.x on PyPI)

1. Confirm root `pyproject.toml` sets **open-fdd** to the intended `2.x` version.
2. Merge/push release commits to **master** (or your release branch).
3. Tag `open-fdd-vX.Y.Z` and push the tag.
4. Confirm [open-fdd on PyPI](#) shows `X.Y.Z`.

40.3.2 PyPI upload auth (open-fdd only)

CI uses **PyPI Trusted Publishing** (OIDC from GitHub). Configure **one** trusted publisher on the `open-fdd` PyPI project:

1. `pypi.org` → project `open-fdd` → **Manage project** → **Publishing** (not Settings).
2. Add a **GitHub** trusted publisher:
 - **Owner:** `bbartling`
 - **Repository:** `open-fdd`
 - **Workflow name:** `publish-open-fdd.yml`
3. Save.

Workflow file: `.github/workflows/publish-open-fdd.yml`. Tags `open-fdd-v*` trigger upload. `workflow_dispatch` is build/check only (no upload) if configured that way.

Official guide: [Publishing with GitHub Actions OIDC](#).

40.3.3 Fallback: API token

If you cannot use OIDC, pass a project-scoped token, for example:

```
uses: pypa/gh-action-pypi-publish@release/v1
with:
  packages-dir: dist/
  password: ${ secrets.PYPI_OPENFDD_TOKEN }
```

40.3.4 If CI shows invalid-publisher or 403

- Trusted publisher on PyPI doesn't match the **exact** workflow filename or repository.
 - Tag points to a commit **before** the OIDC workflow existed — merge fix, retag.
 - **Version already on PyPI** — bump version and use a **new** tag.
-

40.4 2) Publish open-fdd (commands)

40.4.1 Local dry run

```
cd /path/to/open-fdd    # repo root
python -m pip install --upgrade pip build twine
python -m build
twine check dist/*
```

40.4.2 GitHub Actions

```
git checkout master    # or your release branch
# bump version in pyproject.toml, commit, push
git tag open-fdd-v2.0.11
git push origin open-fdd-v2.0.11
```

40.5 3) Publish openfdd-engine (second PyPI project)

1. Create project **openfdd-engine** on PyPI (if it does not exist).
 2. Add a **trusted publisher** pointing at workflow **publish-openfdd-engine.yml** (same repository as open-fdd).
 3. Tag **openfdd-engine-vX.Y.Z** (same numbers as **open-fdd-vX.Y.Z**) and push — only after PyPI is configured, or OIDC fails with invalid-publisher.
-

40.6 Scope policy

- **PyPI** ships the **Python packages** (`open-fdd` is the supported public name for 2.x).
- **Full AFDD stack** (Docker, APIs, graph tooling) lives in **open-fdd-afdd-stack**.

See also: [Engine-only deployment and external IoT pipelines](#).

41 Mode-aware runbooks

42 Mode-aware runbooks

This repository is the `open_fdd rules engine` only. There is no `--mode collector|model|engine|full` Docker orchestration in-tree.

42.1 Recommended checks

Goal	Command
Verify install	<code>python -c "import open_fdd; print('ok')"</code>
Run unit tests	<code>pytest</code>
Editable dev install	<code>pip install -e ".[dev]"</code>

42.2 Test bench vs production data

Treat **synthetic** or **lab** CSVs as a different **operational state** from **production** BMS exports: tighten tolerances, window sizes, and alert routing in production. See [Operational states](#).

43 Testing plan

44 Testing plan

Engine-focused testing for the `open-fdd` package.

44.1 Continuous integration

On every PR and push to **main**, **master**, or **develop**:

1. Install **pip install -e ".[dev]"**
 2. Build combined docs text (python scripts/build_docs_pdf.py --no-pdf) for consistency checks
 3. Run **pytest** on open_fdd/tests
 4. Dry-run **python -m build + twine check** for open-fdd and packages/openfdd-engine
-

44.2 Local hygiene

- Add **pytest** cases for new rule types, edge cases in expressions, and column-map behavior.
 - Keep **fixtures** small and deterministic under **open_fdd/tests/fixtures/rules/**.
-

44.3 Optional benches

If you maintain a separate hardware or BACnet lab, keep its automation **outside** this repository and pass normalized **DataFrames** into **RuleRunner** here.

45 Operations

46 Operations

Engine-focused notes for tests and runbooks. **Deployment, integrity sweeps, and lab automation** for the full stack are documented in **open-fdd-afdd-stack — docs/**.

Page	Description
Testing plan	What CI runs; local pytest expectations.
Mode-aware runbooks	Rules engine vs custom orchestration (no Docker stack in-tree).

47 Configuration

48 Configuration

This page describes how to **configure YAML rules** and engine behavior when using the `open_fdd` library directly. There is **no** required platform database or HTTP service in this repository.

48.1 Rule YAML

Rules are loaded with `load_rule()` / `RuleRunner` (see [Rules overview](#)).

Typical fields include:

Field	Role
<code>name, description</code>	Human-readable metadata
<code>equipment_type</code>	Optional filter for which equipment a rule applies to
<code>type</code>	Check type (bounds, flatline, expression, ...)
<code>params</code>	Check-specific parameters (thresholds, rolling_window, ...)
<code>column_map</code>	Mapping from logical point names to DataFrame columns (dict or manifest path)

Use `fdd_strict_rules`-style tightening in **your** caller if you want stricter validation of column maps and dtypes before evaluation (see [Expression rule cookbook](#)).

48.2 Column maps

- **Inline dict** — simplest: `{ "SAT": "SupplyAirTemp" }` keys match placeholders in the rule.
 - **Manifest YAML** — see [Column map resolvers](#) for `ManifestColumnMapResolver` and composite patterns.
-

48.3 Environment variables

The **open-fdd** wheel does **not** read a fixed `OFDD_*` namespace. Your **application** may use env vars to locate rule directories or data files; that is entirely under your control.

48.4 Related topics

- [Rules overview](#)
- [Engine-only / IoT](#)
- [Getting started](#)

49 Contributing

50 Contributing to Open-FDD

First off, thanks for taking the time to contribute!

Open-FDD is in **Alpha**. The most valuable contributions right now are **bug reports** and **FDD rules**—especially from mechanical engineers and building professionals who can add and refine fault-detection rules using the [Expression Rule Cookbook](#). All types of contributions are encouraged.

Phase focus: Alpha emphasizes **rule correctness**, **engine APIs**, and **documentation** for integrators embedding **open_fdd** in their own data pipelines. Beta (planned) will broaden cookbook coverage and optional modeling guides. See the [Table of Contents](#) for different ways to help. Please read the relevant section before contributing; it helps maintainers and keeps things smooth for everyone.

If you like the project but don't have time to contribute, that's fine. Other ways to support it: - Star the [repository](#) - Share it with colleagues or at meetups - Refer to Open-FDD in your project's readme or documentation

50.1 Table of Contents

- [Code of Conduct](#)
- [I Have a Question](#)
- [I Want To Contribute](#)

- [Reporting Bugs](#)
 - [Suggesting Enhancements](#)
 - [Contributing Rules \(Expression Rule Cookbook\)](#)
 - [Your First Code Contribution](#)
 - [Improving the Documentation](#)
 - [Styleguides](#)
 - [Git workflow \(branch → PR → sync\)](#)
 - [Commit Messages](#)
 - [Join the Project Team](#)
-

50.2 Code of Conduct

This project expects everyone to be respectful and constructive. By participating, you agree to uphold a positive environment. Please report unacceptable behavior by opening an issue or contacting the repository maintainers.

50.3 I Have a Question

Before asking, read [this site's documentation](#) (pip install open-fdd), and search [issues](#). Questions about the **deployed AFDD stack** (API, graph, compose) belong with [open-fdd-afdd-stack](#) and its issue tracker when appropriate.

If you still need help:

- Open an [issue](#).
- Provide as much context as you can (what you're trying to do, what you ran, what happened).
- Include relevant versions: Python, OS, and **open-fdd** release or git commit if known.

We'll respond as soon as we can.

50.4 I Want To Contribute

50.4.1 Legal notice

When contributing, you agree that you have authored 100% of the content, have the necessary rights to it, and that your contribution may be provided under the project license (MIT).

50.5 Reporting Bugs

Bug testing is a top priority in Alpha. Good bug reports help us fix issues quickly.

50.5.1 Before submitting a bug report

- Use the latest version (main branch or latest release).
- Confirm the bug is in **open-fdd** (the rules engine) and not in your environment (e.g. wrong Python version, bad `column_map`). Check the [documentation](#) and [I Have a Question](#) first.
- Search [issues](#) to see if the bug is already reported.
- Collect:
 - **Stack trace** (Traceback) if applicable
 - **OS and platform** (e.g. Linux, macOS, Windows; x86, ARM)
 - **Versions**: Python and how you ran the engine (`pip install`, `editable install`, etc.)
 - **Steps to reproduce** and, if possible, sample input/config
 - Whether you can reproduce it every time and on an older version

50.5.2 How to submit a good bug report

- **Security issues**: Do **not** report security vulnerabilities in public issues. Email the repository owner or open a private security advisory on GitHub.
- Open a new [issue](#). Don't assume it's a bug yet—avoid using the word “bug” in the title until it's confirmed.
- Describe **expected behavior** vs **actual behavior**.
- Provide **reproduction steps** so someone else can recreate the issue. Isolate the problem when possible (e.g. minimal rule YAML, minimal config).
- Paste the information you collected above.

After you file:

- Maintainers will label the issue. If we can't reproduce it, we may ask for more steps and mark it `needs-repro`. Reproducible issues may be marked `needs-fix` and opened for implementation.

50.6 Suggesting Enhancements

Enhancements are tracked as [GitHub issues](#).

50.6.1 Before submitting

- Use the latest version and read the [documentation](#) to see if the behavior already exists or can be configured.
- Search [issues](#) to see if the enhancement was already suggested; if so, add to that discussion.
- Consider whether the idea fits **this repository's scope**: the `open_fdd` rules engine (YAML rules, `RuleRunner`, `column_map`, expression cookbook patterns). Platform-wide features (HTTP services, databases, full Brick graph pipelines) may belong in [open-fdd-afdd-stack](#) instead. Make a clear case for why the change would help most library users.

50.6.2 How to submit a good enhancement suggestion

- Use a **clear, descriptive title**.
 - Describe the **current behavior** and what you **want instead**, and why.
 - Provide **step-by-step detail** where useful; screenshots or example YAML/config can help.
 - Explain why this would be useful to the community.
-

50.7 Contributing Rules (Expression Rule Cookbook)

We especially welcome contributions from mechanical engineers and building professionals who can add or improve FDD rules.

- **Where rules live:** [Fault rules overview](#) — keep project rules in a **YAML directory** you pass to `RuleRunner`. Reload by constructing the runner again after disk edits.
- **How to write rules:** [Expression Rule Cookbook](#) — expression-type rules use YAML with BRICK-style inputs, params, and pandas/NumPy expressions. The cookbook includes AHU-style rules (e.g. GL36-inspired) and patterns you can adapt.
- **What to contribute:**
 - **New rule YAMLs** for common faults (AHU, VAV, plant, sensors) that others can reuse.
 - **Improvements to existing cookbook rules** (thresholds, logic, descriptions).
 - **New cookbook sections or examples** (e.g. for a specific equipment type or standard).

To contribute a rule or cookbook change:

1. Add or edit YAML under **examples/**, **open_fdd/tests/fixtures/rules/**, or your own `rules_dir` for new/updated rules; edit `docs/expression_rule_cookbook.md` (and related docs) for cookbook content.
 2. Follow the existing YAML style and BRICK naming used in the cookbook.
 3. In a PR, describe the fault the rule targets and any assumptions (e.g. Brick types, units). If it's from a standard (e.g. ASHRAE), note the reference.
-

50.8 Your First Code Contribution

- Look for issues labeled `good first issue` or `help wanted` (if we've added them).
 - The codebase targets **Python 3.9+** (see `pyproject.toml`), with **pandas** / **NumPy** at the core. See [Getting started](#) and the README for setup.
 - For rule or engine changes, run **pytest** and, when possible, a small **RuleRunner** smoke test on sample CSV data before submitting a PR.
-

50.9 Improving the Documentation

Documentation for **this repository** lives in `docs/` and is published at bbartling.github.io/open-fdd (Just the Docs — **rules engine** focus). Improvements are welcome:

- Fix typos, clarify wording, or update steps.
- Add examples or how-tos (e.g. for the [Expression rule cookbook](#), [How-to guides](#), or [Column map resolvers](#)).
- Keep code blocks and links in sync with the codebase.

Building a PDF: To generate a single PDF of the docs (e.g. for offline use or to push to the repo), run `python3 scripts/build_docs_pdf.py`. Requires [pandoc](#) and either **weasyprint** (`pip install weasyprint`) or LaTeX. Output is `pdf/open-fdd-docs.pdf` by default; use `--output path` to change it. You can commit and push the PDF if you want it in the repo.

Open a PR with your changes; for large edits, an issue first can help align with maintainers.

After changing README or high-traffic doc links, spot-check the live site (GitHub Pages: extensionless URLs like /getting_started work; a **trailing slash** on leaf pages can 404 depending on hosting).

```
BASE=https://bbartling.github.io/open-fdd
curl -sS -o /dev/null -w "%{http_code}" "$BASE/getting_started"
curl -sS -o /dev/null -w "%{http_code}" "$BASE/
column_map_resolvers"
curl -sS -o /dev/null -w "%{http_code}" "https://github.com/
bbartling/open-fdd/blob/master/pdf/open-fdd-docs.pdf" # expect
302 or 200
```

50.10 Styleguides

- **Python:** We use Black for formatting. Run black on changed files before committing.
 - **YAML:** Match the style in `examples/AHU/rules/` and the expression rule cookbook (indentation, key order).
 - **Markdown:** Use clear headings and lists; link to existing docs where relevant.
-

50.11 Git workflow (branch → PR → sync)

Create the branch **before** committing so that after the PR is merged, git pull on main/master fast-forwards and you avoid divergent-branch errors (protected main/master).

1. Create a branch

```
git checkout -b feature/short-name
```

2. Stage, commit, and push

```
git add .
git commit -m 'short note describing the change'
git push -u origin feature/short-name
```

3. Open a PR manually on GitHub from feature/short-name into main (or master). Create the PR in the GitHub UI; we do not use a CLI for this step.

4. After the PR is merged — fetch prune, switch to default branch, and pull to sync:

```
git fetch --prune
git checkout main
git pull
```

Use master instead of main if that is your default branch. If you ever committed on main/master before creating the branch, local and remote can diverge after the merge; run `git reset --hard origin/main` (or `origin/master`) to match the remote.

50.12 Commit Messages

- Use a clear, short summary line. Optionally add a body with details.
 - Reference issues/PRs when relevant (e.g. `Fix bounds rule when input is all NaN (#123)`).
-

50.13 Join the Project Team

If you're interested in ongoing maintenance or a larger role, say so in an issue or reach out to the maintainers. We're especially interested in folks who can triage bugs, review rule contributions, or improve docs and the cookbook.

50.14 Attribution

This contributing guide was inspired by the [contributing.md](#) project. You don't need to pay to create or use a CONTRIBUTING file—it's just a markdown file in your repo that helps contributors and maintainers.

51 AI PR review playbook

52 AI PR review playbook

Use this checklist when an AI agent reviews Open-FDD pull requests.

52.1 Priority order

1. Behavior regressions and runtime risks.
2. Auth/security exposure or secret handling mistakes.
3. Data-model/API contract breaks.
4. Missing tests for changed behavior.
5. Documentation drift for operationally meaningful changes.

52.2 Review output contract

- Findings first, ordered by severity.
- Include concrete file references and reproducible rationale.
- Keep summary brief after findings.
- If no findings, state residual risks/testing gaps explicitly.

53 Appendix

54 Appendix

Technical deep dives and maintainer reference material.

Page	Description
API Reference	Python engine entry points; points to Engine API .
Technical reference	Repository layout, tests, and maintainer notes.
Developer guide	Contributing workflow for the open_fdd package.
AI PR review playbook	Review checklist focused on regressions, risks, and missing validation before merge.

55 Cloning and porting

56 Cloning and porting

The **open-fdd** repository should be portable across workstations and CI images: clone, create a virtual environment, `pip install -e ".[dev]"`, run `pytest`.

56.1 What transfers cleanly

- Automated tests (open_fdd/tests, CI in .github/workflows/ci.yml)
- Example rules under **examples/** and **open_fdd/tests/fixtures/rules/**
- Documentation under **docs/**

56.2 What changes per deployment

- Paths to CSV or Parquet inputs
- **column_map** from your naming convention to DataFrame columns
- Optional Brick TTL or SQL metadata **you** maintain outside this package

56.3 Recommended first pass on a new machine

1. `git clone` and `pip install -e ".[dev]"`
2. `python -c "import open_fdd; print('open_fdd OK')"`
3. `pytest`
4. Run a small example from **examples/README.md**

56.4 Portability goal

Another engineer should be able to reproduce rule behavior from **version-controlled YAML** and a **documented data sample**, without private containers or undisclosed APIs.

57 Verification

58 Verification

Practical checks when authoring or shipping **open-fdd** rules.

58.1 1. Unit tests (CI)

From the repo root with dev dependencies:

```
pip install -e ".[dev]"
pytest
```

58.2 2. Rule YAML sanity

- Load each file with `load_rule()` or point `RuleRunner(rules_path=...)` at the directory.
 - Confirm every **inputs** key has a matching entry in **column_map** (or manifest) before calling `run()`.
-

58.3 3. Small DataFrame smoke test

```
from pathlib import Path
import pandas as pd
from open_fdd.engine.runner import RuleRunner

df = pd.read_csv("your_sample.csv",
parse_dates=["timestamp"]).set_index("timestamp")
runner = RuleRunner(rules_path=Path("path/to/rules"))
out = runner.run(df, timestamp_col="timestamp",
column_map={"Supply_Air_Temperature_Sensor": "SAT"})
assert out.filter(like="_flag").shape[1] >= 1
```

58.4 Platform / database checks

For HTTP APIs, databases, Brick / 223P graph workflows, and observability around a deployed stack, see **[open-fdd-afdd-stack documentation](#)**.

59 Engine-only deployment and external IoT pipelines

60 Engine-only deployment and external IoT pipelines

Library-only (`pip install open-fdd`): See the **engine documentation** — [Getting started](#), [Column map & resolvers](#), and this page for pandas integrators.

Integrators who already run **data collection** (historians, MQTT, BAS exports) and their own **modeling** can add **FDD** with `open_fdd.engine.RuleRunner` and the same **YAML** rules you would use in any other context.

Package names: The rules code lives under `open_fdd.engine`. The optional `openfdd-engine` package (`openfdd_engine`) is a thin re-export — not a different engine. See [The optional openfdd-engine package](#).

60.1 Library path — same YAML, any DataFrame

The rule runner is `open_fdd.engine.runner.RuleRunner`. It loads `.yaml` rule files (type: `bounds|flatline|expression|...`, inputs, params, ...).
Authoring references:

- [Expression rule cookbook](#)
- Examples under `examples/` in the repository

Minimal integration pattern

1. Build a **pandas** DataFrame whose columns are your sensor traces (and optional timestamp column).
2. Point **RuleRunner** at a directory of `.yaml` files **or** pass `rules=[...]` dicts.
3. Call `run(df, timestamp_col=..., skip_missing_columns=True, column_map={...})`.
4. Read boolean `*_flag` columns and related outputs per the rule definitions.

column_map — when your naming layer uses Brick-style or vendor tags but DataFrame columns differ (for example temp_sa vs a Brick class label), pass **column_map** from logical key to column name. See `RuleRunner.run` in `open_fdd/engine/runner.py`.

60.1.1 Bring your own column_map or resolver

1. **Plain dict** — `column_map: dict[str, str]` (rule input key → DataFrame column). Pass to `RuleRunner.run(..., column_map=column_map)`.
2. **Brick TTL + SPARQL** — The `open-fdd` wheel does **not** include `rdflib`. If you resolve points from TTL yourself, install `rdflib` in your environment and build a dict before calling `RuleRunner`.
3. **Custom ColumnMapResolver** — Implement `build_column_map(*, ttl_path: Path) -> dict[str, str]` per `open_fdd.engine.column_map_resolver`, or use `ManifestColumnMapResolver` / `FirstWinsCompositeResolver` for manifest-driven maps.

Priority / policy: The engine uses **one** `column_map` per run. Resolve ambiguity in your pipeline before calling `RuleRunner`.

60.1.2 Manifest file + composite priority

- `load_column_map_manifest(path)` — reads `.json` or `.yaml`. Accepts a flat object or a nested `column_map`: mapping.
- `ManifestColumnMapResolver(path)` — exposes the manifest via the resolver protocol.
- `FirstWinsCompositeResolver(r1, r2, ...)` — first resolver to supply a key wins.

Examples: `examples/column_map_resolver_workshop/` — `simple_ontology_demo.py`.

Install

```
pip install open-fdd
# or from a checkout:
pip install -e ".[dev]"
```

60.2 Standalone playground (optional)

- `examples/engine_iot_playground/` — README, rules, sample CSV, `run_demo.py`, notebook.
-

60.3 Summary

Approach	You bring	Open-FDD provides
RuleRunner in Python	DataFrame + YAML rules	Fault columns and structured outputs
Batch / notebook	CSV or query results	Same semantics as any other caller

For package layout, see [Modular architecture](#).

61 The optional openfdd-engine package

62 The optional openfdd-engine package (openfdd_engine)

Open-FDD has **one rules implementation**: YAML files evaluated on **pandas** via **RuleRunner** in **open_fdd.engine**. The **packages/openfdd-engine/** tree (PyPI name **openfdd-engine**, import **openfdd_engine**) is a **thin re-export** of selected symbols from **open_fdd.engine** — useful when procurement or legacy imports expect the **openfdd_engine** package name.

62.1 Re-exports only

openfdd_engine does not duplicate engine logic. Its **__init__.py** imports from **open_fdd.engine** and lists them in **__all__**. New public APIs belong in **open_fdd.engine** first; the shim gains them through re-export.

- **Versioning**: **open-fdd** on PyPI is authoritative; **openfdd-engine** pins **open-fdd** `>=...` when published.

62.2 Mental model

Layer	What it is	Typical import
Core engine		from open_fdd.engine.runner import RuleRunner

Layer	What it is	Typical import
	Loads YAML rules, runs checks on a DataFrame	
Optional shim	Same classes, different package name	from openfdd_engine import RuleRunner

62.3 When to use `pip install open-fdd` vs `openfdd-engine`

Default:

```

pip install open-fdd

from open_fdd.engine.runner import RuleRunner, load_rule,
load_rules_from_dir
from open_fdd.engine.column_map_resolver import
load_column_map_manifest

```

Optional `openfdd-engine` — only if you need the separate PyPI project or editable shim:

```

cd packages/openfdd-engine
pip install -e .

```

See [PyPI releases \(open-fdd\)](#).

62.4 Summary

Goal	Install	Import
Normal use	<code>pip install open-fdd</code>	<code>open_fdd.engine.*</code>
Optional second package	published <code>openfdd-engine</code> or editable shim	<code>openfdd_engine.*</code>

62.5 See also

- [Engine-only deployment and external IoT pipelines](#)
- [Modular architecture](#)
- [Expression rule cookbook](#)
- `packages/openfdd-engine/README.md`

63 Desktop App

64 Open-FDD Desktop App

64.1 Goal

Run Open-FDD locally with a Python bridge + MCP + web UI, including container-friendly workflows for OpenClaw.

This repository includes a React UI workspace at `apps/desktop-ui` that talks to a local Python bridge. The recommended automation path is now web-first (bridge + MCP + React UI), which works cleanly in Docker/container environments.

64.2 Install

```
pip install "open-fdd[desktop]"
```

64.3 Launch

64.3.1 Container-friendly web launch (bridge + MCP + React UI)

Recommended launcher on Windows:

```
powershell -ExecutionPolicy Bypass -File .\scripts\bootstrap-desktop.ps1 -InstallDeps
```

Daily startup (without reinstalling deps):

```
powershell -ExecutionPolicy Bypass -File .\scripts\bootstrap-desktop.ps1
```

`bootstrap-desktop.ps1` launches by default: - terminal 1: `open-fdd-desktop-bridge` - terminal 2: `open-fdd-mcp-rag` - terminal 3: React UI (static mode by default) - bridge Swagger: `http://127.0.0.1:8765/docs` - bridge OpenAPI: `http://127.0.0.1:8765/openapi.json` - MCP API: `http://127.0.0.1:8090` - Web UI: `http://127.0.0.1:8080`

```
# terminal 1
open-fdd-desktop-bridge
```

```
# terminal 2
open-fdd-mcp-rag
```

```
# terminal 3
cd apps/desktop-ui
npm install
npm run build
python -m http.server 8080 --directory dist --bind 0.0.0.0
```

Use Vite dev mode instead of static:

```
powershell -ExecutionPolicy Bypass -File .\scripts\bootstrap-
desktop.ps1 -UiMode dev -UiPort 5173
```

Linux/macOS/bash launcher (including container shells):

```
bash ./scripts/bootstrap-desktop.sh --install-deps
```

Daily startup:

```
bash ./scripts/bootstrap-desktop.sh
```

Useful flags:

- --no-bridge
- --no-mcp
- --no-ui
- --no-launch
- --ui-mode static|dev
- --ui-port 8080
- --bridge-url http://127.0.0.1:8765

PowerShell equivalents:

- -NoBridge
- -NoMcp
- -NoUi
- -NoLaunch
- -UiMode static|dev
- -UiPort 8080
- -BridgeUrl http://127.0.0.1:8765

Container note:

- Static mode (UiMode=static / --ui-mode static) is preferred for repeatable container startup.
- Dev mode is useful when actively editing UI code.

Bridge URL consistency:

- Bootstrap sets `OFDD_BRIDGE_URL` for the bridge process and `VITE_DESKTOP_BRIDGE_BASE` for the UI build so both target the same base URL (override with --bridge-url / -BridgeUrl or `OFDD_BRIDGE_URL`).

- The bridge endpoint also reads optional `OFDD_BRIDGE_HOST` / `OFDD_BRIDGE_PORT` if you prefer host/port env vars instead of a full URL.

64.3.2 Desktop bridge Swagger/OpenAPI

Once bridge is running locally:

- Swagger UI: `http://127.0.0.1:8765/docs`
- OpenAPI JSON: `http://127.0.0.1:8765/openapi.json`

Use Swagger for endpoint discovery, request body examples, and quick local API testing for OpenClaw or other assistants.

64.4 MCP RAG service (OpenClaw/agents)

`open-fdd` includes an MCP-style RAG HTTP service that can index local `docs/` and proxy driver/model tools to the desktop bridge API.

Build the local retrieval index:

```
python scripts/build_mcp_rag_index.py --output stack/mcp-rag/index/rag_index.json
```

Run MCP RAG locally:

```
open-fdd-mcp-rag
# serves on http://127.0.0.1:8090
```

Key env vars: - `OFDD_MCP_OFDD_API_URL` (default `http://127.0.0.1:8765`) - `OFDD_MCP_OFDD_API_KEY` - `OFDD_MCP_ENABLE_ACTION_TOOLS=true` (required for write/config/ingest proxy tools) - `OFDD_MCP_RAG_INDEX_PATH` (default `./stack/mcp-rag/index/rag_index.json`)

Docker image scaffold: - `stack/Dockerfile.mcp_rag`

64.5 Data ingest quickstart (desktop bridge API)

Base URL:

- `http://127.0.0.1:8765`

64.5.1 1) Create or list sites

```
curl http://127.0.0.1:8765/sites
curl -X POST http://127.0.0.1:8765/sites -H "Content-Type: application/json" -d "{\"name\":\"HQ\"}"
```

64.5.2 2) CSV ingest (file path or upload)

```
# direct path
curl -X POST http://127.0.0.1:8765/ingest/csv \
  -H "Content-Type: application/json" \
  -d "{\"site_id\":\"<site-id>\",\"source\":\"csv\",\"csv_path\":\"C:/data/site.csv\"}"

# multipart upload
curl -X POST http://127.0.0.1:8765/ingest/csv/upload \
  -F "site_id=<site-id>" \
  -F "source=csv" \
  -F "file=@C:/data/site.csv"
```

64.5.3 3) Open-Meteo weather ingest

```
# set weather config once
curl -X POST http://127.0.0.1:8765/config/weather \
  -H "Content-Type: application/json" \
  -d "{\"latitude\":42.36,\"longitude\":-71.06,\"timezone\":\"America/New_York\",\"base_url\":\"https://archive-api.open-meteo.com/v1/archive\"}"

# ingest weather rows
curl -X POST http://127.0.0.1:8765/ingest/weather \
  -H "Content-Type: application/json" \
  -d "{\"site_id\":\"<site-id>\",\"days_back\":7}"
```

64.5.4 4) Onboard ingest

Prerequisite: configure Onboard first (fresh desktop defaults are usually not enough):

```
curl -X POST http://127.0.0.1:8765/config/onboard \
  -H "Content-Type: application/json" \
  -d "{\"base_url\":\"https://api.onboarddata.io\", \"building_ids\":\"123,456\", \"api_key\":\"<token>\", \"lookback_hours\":24, \"allow_synthetic\":false}"
```

Required onboarding keys are `base_url`, `building_ids`, and `api_key` (either via `/config/onboard` or env vars) before `/ingest/onboard`.

```
curl -X POST http://127.0.0.1:8765/ingest/onboard \
-H "Content-Type: application/json" \
-d "{\"site_id\":\"<site-id>\"}"
```

64.5.5 5) BACnet ingest (one-shot) and polling

```
# one-shot pull
curl -X POST http://127.0.0.1:8765/ingest/bacnet \
-H "Content-Type: application/json" \
-d "{\"site_id\":\"<site-id>\",\"server_url\":\"http://192.168.204.18:8080\",\"api_key\":\"<token>\"}"

# enable 5-minute poll loop
curl -X POST http://127.0.0.1:8765/config/bacnet \
-H "Content-Type: application/json" \
-d "{\"enabled\":true,\"interval_seconds\":300,\"site_id\":\"<site-id>\",\"server_url\":\"http://192.168.204.18:8080\",\"api_key\":\"<token>\"}"
```

64.5.6 6) Join sources for analysis/plots

```
curl -X POST http://127.0.0.1:8765/timeseries/query \
-H "Content-Type: application/json" \
-d "{\"site_id\":\"<site-id>\",\"sources\":[\"csv\",\"weather\",\"onboard\",\"bacnet\"],\"join_on_timestamp\":true,\"join_how\":\"outer\",\"limit\":10000}"
```

64.6 CSV timestamp parsing expectations

Desktop CSV ingest is strict enough to catch bad files but flexible on common field formats:

- Timestamp column auto-detection prefers timestamp, then other time/date-like headers.
- Known timezone abbreviations like EDT, EST, CDT, PDT, etc. are normalized to UTC offsets before parsing.
- Parsing uses UTC-normalized datetimes and drops rows with unparseable timestamps.
- A minimum valid timestamp ratio is enforced; files with mostly bad timestamps return a clear validation error instead of silently importing junk data.

Recommended CSV practice:

- Include a dedicated timestamp column (timestamp preferred).
- Use ISO 8601 when possible (example: 2026-03-18T21:00:00-04:00).

- Avoid mixed timestamp formats inside one file.

If import fails, fix timestamp formatting and retry.

64.7 Data model + BRICK

- Desktop stores model data in user-writable app-data (open-fdd-desktop/model.json).
- BRICK TTL is generated to open-fdd-desktop/data_model.ttl.
- App-data root is platform-specific (%APPDATA% on Windows, ~/.local/share on Linux, ~/Library/Application Support on macOS) and is resolved by `open_fdd.desktop.storage.paths.desktop_data_dir`.
- BRICK input mapping for rules resolves through `open_fdd.desktop.services.BrickService`.

64.8 Feather-first ingestion

- CSV, weather, onboard, and BACnet drivers write pandas frames into timestamped Feather files under open-fdd-desktop/feather_store.
- Feather path layout is source/site scoped:
 - open-fdd-desktop/feather_store/<safe_source>/<safe_site_id>/<timestamp>_<nonce>.feather
- Storage remains append/chunk-based (many files per site/source) for reliability and backfill workflows.
- The desktop bridge can still present a site-level joined view for plotting (multi-source virtual merge by timestamp), so operators get a single logical trend frame without rewriting raw files.
- Time-series reads for rules concatenate all Feather files for a selected (source, site_id) pair.
- Point metadata supports external refs like:
 - feather://<source>/<site_id>/<metric>
- These refs are persisted in the model and emitted in TTL via `ofdd:externalReference`.

64.9 Rule loop behavior

- `open_fdd.desktop.rules.run_rule_loop_batched` executes rules with memory-aware chunking.
- If data fits available memory target, full-frame evaluation is used.
- Otherwise, DataFrames are chunked and processed with equivalent rule semantics.
- Chunk size can be user-forced from desktop (`chunk_rows`) or auto-estimated from available memory.

- Batched outputs are concatenated back into one result frame so downstream fault summaries behave the same as single-pass runs.

64.10 Typical desktop workflow (current)

1. Create/select a site in the desktop model tab.
2. Ingest data (CSV import, weather fetch, or onboard scrape) into Feather-backed storage.
3. Confirm points and external references in model + generated BRICK TTL.
4. Run rules from a local YAML rules directory.
5. For large datasets, use batched rule execution (`chunk_rows > 0`, or leave auto-estimation on).
6. Review fault columns and summaries from the merged output frame.

64.11 Development

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -U pip
pip install -e ".[dev,desktop]"
pytest
```